HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: IoT & ứng dụng

MÃ HỌC PHẦN: INT14149

B23DCAT140 Ngô Thị Thanh Huyền

Tên nhóm: 12

Giảng viên hướng dẫn: TS.Nguyễn Quốc Uy

HÀ NỘI 8-2026

MỤC LỤC

MỤC LỤC 2
DANH MỤC CÁC HÌNH ẢNH 4
CHƯƠNG 1. GIỚI THIỆU TỔNG QUAN 5
1.1 Đặt vấn đề 5
1.2 Mục tiêu của đề tài 5
1.3 Đối tượng và phạm vi nghiên cứu 6
1.3.1 Đối tượng nghiên cứu 6
1.3.2 Phạm vi nghiên cứu 6
1.4 Phương pháp nghiên cứu 7
1.5 Bố cục báo cáo 7
CHƯƠNG 2. Thiết kế hệ thống (System Design) 8
2.1 Kiến trúc tổng thể hệ thống (System Architecture) 8
2.1.1 Lớp thiết bị ngoại vi (Perception / Device Layer) 8
2.1.2 Lớp mạng và Giao thức truyền thông (Network & Protocol Layer) 10
2.1.3 Lớp Xử lý và Lưu trữ (Backend & Database Layer) 11
2.1.4 Lớp Ứng dụng và Giao diện (Application / UI Layer) 11
2.2 Cơ sở dữ liệu 12
CHƯƠNG 3. Thiết kế chi tiết 14
3.1 *Đặc tả usecase* 14
3.1.1 Use case: Đăng nhập hệ thống(Login) 14
Mô tả sequence: 15
3.1.2 Use case: Giám sát dữ liệu thời gian thực (Dashboard) 17
3.1.3 Use case: Tra cứu và lọc dữ liệu cảm biến (Data Sensor) 20
3.1.4 Use case:Device control(Điều khiển thiết bị) 22
3.1.5 Use case: Xem lịch sử hoạt động (Action History) 24
3.1.6 Use case: Xem thông tin cá nhân (Profile) 26
3.2 API 29
CHƯƠNG 4. KẾT QUẢ THỰC HIỆN 30
4.1 Chức năng Đăng nhập hệ thống (Authentication) 30
4.2 Màn hình Dashboard giám sát thời gian thực 30
4.3 Chức năng Điều khiển thiết bị & Hiệu ứng nút xoay Loading 31
4.4 Cơ chế Cảnh báo mất tín hiệu phần cứng (Hardware Watchdog) 32
4.5 Quản lý Dữ liệu cảm biến (DataSensor) với Phân trang thông minh 33

4.6 Quản lý Lịch sử thao tác (Action History) 34
4.7 Trang Hồ sơ cá nhân (Profile) & Tích hợp liên kết 34
4.8 Cơ sở dữ liệu MySQL trên XAMPP 36
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 36
5.1 Kết luận 36
5.2 Hạn chế của đề tài 37
5.3 Hướng phát triển trong tương lai 38

DANH MỤC CÁC HÌNH ẢNH

Hình 1 ESP8266 9
Hình 2 Cảm biến DHT11 9
Hình 3 Quang trở 10
Hình 4 Hệ thống trao đổi giữa phần cứng-lớp mạng-ứng dụng&giao diện 12
Hình 5 Biểu đồ sequence cho đăng nhập hệ thống 15
Hình 6 Giao diện Login 17
Hình 7 Biểu đồ sequence của usecase Giám sát dữ liệu thời gian thực 18
Hình 8 Giao diện Dashboard 20
Hình 9 Biểu đồ sequence tra cứu và lọc dữ liệu cảm biến 21
Hình 10 Giao diện Datasensor 22
Hình 11 Biểu đồ sequence điều khiển thiết bị 23
Hình 12 Biểu đồ sequence xem lịch sử hoạt động 25
Hình 13 Giao diện Action History 26
Hình 14 Biểu đồ sequence xem thông tin cá nhân 27
Hình 15 Giao diện Profile 28
Hình 16 Giao diện đăng nhập hệ thống 30
Hình 17 Giao diện Dashboard hiển thị số đo thực tế và biểu đồ 31
Hình 18 Trạng thái khi che cảm biến quang trở]) 31
Hình 19 Hiệu ứng nút xoay Loading khi đang điều khiển thiết bị 32
Hình 20 Trạng thái thiết bị sau khi chuyển mạch thành công 32
Hình 21 Banner cảnh báo trên màn hình khi ngắt kết nối phần cứng]) 33
Hình 22 Bảng dữ liệu DataSensor và thanh phân trang thu gọn dạng ...]) 34
Hình 23 Bảng nhật ký điều khiển Action History 34
Hình 24 Giao diện trang Profile cá nhân]) 35
Hình 25 Popup chỉnh sửa thông tin cá nhân và gắn link Postman]) 35
Hình 26 Giao diện phpMyAdmin trên XAMPP thể hiện dữ liệu trong bảng datasensor và action_history 36

CHƯƠNG 1. GIỚI THIỆU TỔNG QUAN

1.1 Đặt vấn đề

Trong bối cảnh cuộc Cách mạng công nghiệp lần thứ tư đang phát triển mạnh mẽ, Internet vạn vật (Internet of Things – IoT) ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực của đời sống như sản xuất, y tế, nông nghiệp và quản lý nhà ở. Công nghệ IoT cho phép các thiết bị vật lý kết nối với mạng, thu thập dữ liệu từ môi trường, trao đổi thông tin và thực hiện các hoạt động điều khiển từ xa. Nhờ đó, việc giám sát và quản lý các thiết bị trở nên thuận tiện, linh hoạt và hiệu quả hơn.

Một trong những ứng dụng tiêu biểu của IoT là hệ thống nhà thông minh (Smart Home). Hệ thống này cho phép người dùng theo dõi các thông số môi trường trong nhà, chẳng hạn như nhiệt độ, độ ẩm và cường độ ánh sáng, đồng thời điều khiển các thiết bị điện thông qua giao diện trực quan. Việc ứng dụng IoT vào quản lý nhà ở không chỉ góp phần nâng cao sự tiện nghi mà còn tạo tiền đề cho việc tự động hóa các hoạt động sinh hoạt, tiết kiệm năng lượng và quản lý thiết bị hiệu quả hơn.

Tuy nhiên, để xây dựng một hệ thống nhà thông minh hoạt động hiệu quả, cần kết hợp nhiều thành phần khác nhau, bao gồm thiết bị phần cứng, cảm biến, giao thức truyền thông, máy chủ xử lý, cơ sở dữ liệu và giao diện người dùng. Trong đó, việc bảo đảm quá trình truyền dữ liệu ổn định, lưu trữ thông tin có hệ thống và phản hồi chính xác các lệnh điều khiển là những vấn đề quan trọng cần được quan tâm.

Xuất phát từ những yêu cầu trên, đề tài "Xây dựng hệ thống nhà thông minh ứng dụng công nghệ IoT" được thực hiện nhằm nghiên cứu và triển khai mô hình giám sát môi trường, điều khiển thiết bị điện thông qua mạng cục bộ. Hệ thống sử dụng vi điều khiển ESP8266 để thu thập dữ liệu từ cảm biến nhiệt độ – độ ẩm DHT và cảm biến ánh sáng LDR; sử dụng giao thức MQTT để trao đổi dữ liệu giữa thiết bị phần cứng và máy chủ; đồng thời kết hợp cơ sở dữ liệu MySQL và giao diện Web để hiển thị thông số, tra cứu lịch sử và điều khiển thiết bị.

Thông qua quá trình thực hiện đề tài, người thực hiện có cơ hội vận dụng kiến thức về lập trình nhúng, giao thức truyền thông IoT, phát triển ứng dụng Web và thiết kế cơ sở dữ liệu vào một mô hình thực tế. Đây cũng là cơ sở để nghiên cứu, đánh giá khả năng mở rộng hệ thống trong tương lai.

1.2 Mục tiêu của đề tài

Đề tài hướng đến xây dựng một mô hình nhà thông minh có khả năng thu thập dữ liệu môi trường, hiển thị thông tin trên giao diện Web và hỗ trợ điều khiển thiết bị từ xa. Các mục tiêu cụ thể bao gồm:

Thứ nhất, về phần cứng: Sử dụng vi điều khiển ESP8266 kết nối với cảm biến DHT để thu thập nhiệt độ, độ ẩm và cảm biến quang trở LDR để ghi nhận sự thay đổi của ánh sáng môi trường. Đồng thời, tích hợp đèn LED hoặc cơ cấu chấp hành phù hợp để thực hiện thao tác bật, tắt thiết bị theo lệnh điều khiển.

Thứ hai, về truyền thông và xử lý dữ liệu: Xây dựng cơ chế trao đổi dữ liệu giữa ESP8266 và máy chủ thông qua giao thức MQTT. Máy chủ tiếp nhận dữ liệu cảm biến, xử lý thông tin, cung cấp các API phục vụ giao diện Web và hỗ trợ gửi lệnh điều khiển đến thiết bị phần cứng.

Thứ ba, về cơ sở dữ liệu: Thiết kế cơ sở dữ liệu quan hệ MySQL để quản lý tài khoản người dùng, danh mục cảm biến, danh mục thiết bị, dữ liệu cảm biến và lịch sử điều khiển. Dữ liệu được tổ chức nhằm phục vụ việc truy vấn, thống kê và theo dõi hoạt động của hệ thống.

Thứ tư, về giao diện người dùng: Xây dựng giao diện Web trực quan, cho phép người dùng theo dõi các thông số môi trường trên Dashboard, quan sát biểu đồ dữ liệu, tra cứu lịch sử cảm biến, điều khiển thiết bị và xem lịch sử thao tác.

Thứ năm, về giám sát trạng thái hệ thống: Xây dựng cơ chế theo dõi tín hiệu từ phần cứng, phát hiện tình trạng mất kết nối dựa trên thời gian không nhận được dữ liệu và thông báo cho người dùng khi thiết bị ngoại tuyến. Đối với thao tác điều khiển, hệ thống cần thể hiện trạng thái đang xử lý và kết quả phản hồi từ thiết bị.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là mô hình nhà thông minh ứng dụng công nghệ IoT, tập trung vào quá trình thu thập dữ liệu cảm biến, truyền nhận thông tin, lưu trữ dữ liệu và điều khiển thiết bị thông qua giao diện Web.

Các thành phần chính được nghiên cứu và sử dụng trong hệ thống gồm:

- Vi điều khiển ESP8266 có khả năng kết nối Wi-Fi.
- Cảm biến nhiệt độ – độ ẩm DHT.
- Cảm biến quang trở LDR để ghi nhận sự thay đổi của ánh sáng.
- Đèn LED hoặc thiết bị chấp hành phục vụ chức năng điều khiển.
- Giao thức MQTT phục vụ trao đổi dữ liệu giữa phần cứng và máy chủ.
- Máy chủ Backend, cơ sở dữ liệu MySQL và giao diện Web Dashboard.

1.3.2 Phạm vi nghiên cứu

Trong khuôn khổ bài tập lớn học phần IoT & ứng dụng, đề tài tập trung triển khai các chức năng chính sau:

- Giám sát môi trường: Thu thập và hiển thị ba thông số gồm nhiệt độ, độ ẩm và giá trị ánh sáng từ cảm biến.
- Hiển thị dữ liệu: Trình bày các thông số mới nhất và biểu đồ biến thiên trên Dashboard.
- Điều khiển thiết bị: Cho phép người dùng gửi lệnh bật, tắt đèn từ giao diện Web và theo dõi trạng thái xử lý lệnh.

- Quản lý dữ liệu cảm biến: Lưu trữ, tra cứu và lọc dữ liệu theo các điều kiện phù hợp.
- Quản lý lịch sử điều khiển: Ghi nhận thiết bị được thao tác, hành động, trạng thái xử lý và thời gian thực hiện.
- Quản lý tài khoản và thông tin cá nhân: Hỗ trợ đăng nhập và hiển thị thông tin người dùng theo các chức năng được xây dựng.
- Giám sát kết nối: Phát hiện tình trạng không nhận được tín hiệu từ phần cứng trong khoảng thời gian quy định và hiển thị cảnh báo trên giao diện.

Hệ thống được triển khai thử nghiệm trong môi trường mạng cục bộ (LAN), kết nối giữa máy tính chạy các thành phần máy chủ và thiết bị ESP8266. Đề tài chưa tập trung triển khai trên quy mô nhà ở thực tế với nhiều phòng và nhiều loại thiết bị gia dụng.

1.4 Phương pháp nghiên cứu

Để thực hiện các mục tiêu đã đề ra, đề tài sử dụng kết hợp phương pháp nghiên cứu lý thuyết và phương pháp thực nghiệm.

Phương pháp nghiên cứu lý thuyết: Tìm hiểu các kiến thức cơ bản về IoT, vi điều khiển ESP8266, nguyên lý hoạt động của cảm biến DHT và LDR, giao thức MQTT, mô hình giao tiếp giữa Frontend và Backend, cũng như phương pháp tổ chức dữ liệu trong hệ quản trị cơ sở dữ liệu MySQL.

Phương pháp thiết kế hệ thống: Phân tích yêu cầu chức năng, xác định các thành phần của hệ thống, xây dựng kiến trúc tổng thể, thiết kế cơ sở dữ liệu và mô tả luồng trao đổi thông tin giữa thiết bị, máy chủ và giao diện Web.

Phương pháp thực nghiệm: Lắp đặt và lập trình phần cứng, xây dựng các thành phần phần mềm, kết nối các thành phần qua mạng cục bộ và kiểm tra hoạt động của hệ thống thông qua những tình huống cụ thể như thay đổi điều kiện ánh sáng, cập nhật thông số cảm biến, bật/tắt đèn và mất kết nối phần cứng.

Phương pháp đánh giá: Quan sát kết quả thực tế, đối chiếu hoạt động của hệ thống với các mục tiêu đề ra, phát hiện những hạn chế trong quá trình vận hành và đề xuất phương án cải thiện.

1.5 Bố cục báo cáo

Ngoài phần mở đầu và các tài liệu tham khảo, báo cáo được tổ chức thành năm chương với nội dung chính như sau:

- **Chương 1. Giới thiệu tổng quan:** Trình bày lý do lựa chọn đề tài, mục tiêu, đối tượng, phạm vi và phương pháp nghiên cứu.
- **Chương 2. Thiết kế hệ thống:** Trình bày kiến trúc tổng thể, các thành phần phần cứng, giao thức truyền thông và thiết kế cơ sở dữ liệu.
- **Chương 3. Thiết kế chi tiết:** Mô tả các chức năng của hệ thống, luồng xử lý nghiệp vụ và các API phục vụ hoạt động của ứng dụng.

- **Chương 4. Kết quả thực hiện:** Trình bày kết quả triển khai các chức năng, giao diện, cơ sở dữ liệu và quá trình kiểm tra hệ thống.
- **Chương 5. Kết luận và hướng phát triển:** Tổng kết kết quả đạt được, đánh giá những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 2. THIẾT KẾ HỆ THỐNG (SYSTEM DESIGN)

2.1 Kiến trúc tổng thể hệ thống (System Architecture)

Hệ thống được thiết kế dựa trên mô hình kiến trúc 3 lớp (3-Tier Architecture) tiêu chuẩn của IoT, bao gồm: Lớp thu thập dữ liệu (Thiết bị), Lớp xử lý trung tâm (Backend & Database), và Lớp giao diện tương tác (Frontend). Việc phân tách này giúp hệ thống dễ dàng mở rộng (scalability), bảo trì độc lập và đảm bảo hiệu suất thời gian thực.

2.1.1 Lớp thiết bị ngoại vi (Perception / Device Layer)

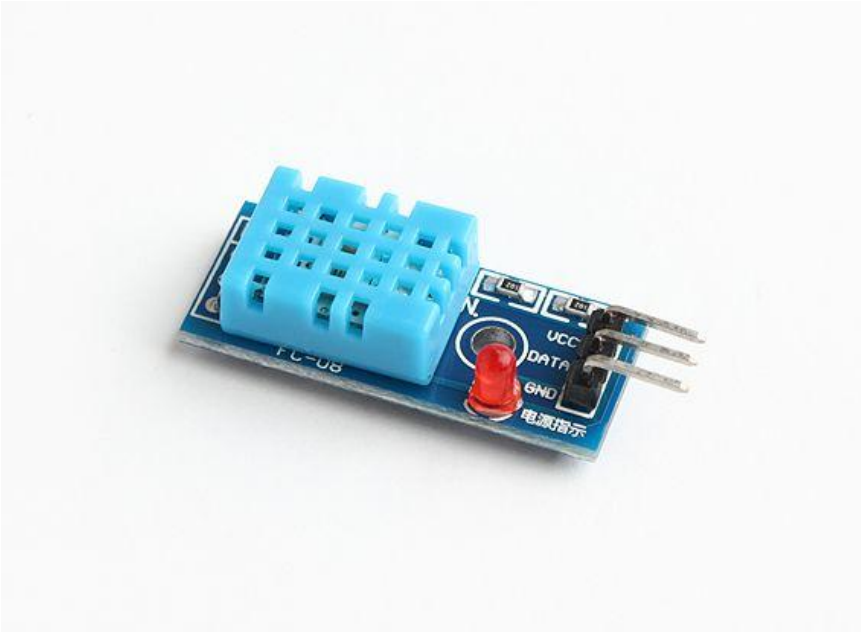
Lớp này đóng vai trò là "giác quan" và "tay chân" của toàn bộ hệ thống, chịu trách nhiệm thu thập thông số môi trường liên tục và thực thi các lệnh vật lý.

- **Vi điều khiển trung tâm (ESP8266):**
    - Là vi mạch Wi-Fi SoC (System on a Chip) tích hợp vi xử lý Tensilica L106 32-bit.
    - Hỗ trợ chuẩn Wi-Fi 802.11 b/g/n, cho phép kết nối trực tiếp với router mạng nội bộ để giao tiếp với Internet mà không cần thông qua Gateway trung gian phức tạp.
    - Cung cấp các chân GPIO (General Purpose Input/Output) và bộ chuyển đổi ADC (Analog-to-Digital Converter) 10-bit để giao tiếp đa dạng với các loại cảm biến digital và analog.



Hình 1 ESP8266

- **Cảm biến Nhiệt độ & Độ ẩm (DHT11 / DHT22):**
    - Hoạt động dựa trên nguyên lý đo điện trở của vật liệu hút ẩm (đối với độ ẩm) và nhiệt trở NTC (đối với nhiệt độ).
    - Sử dụng chuẩn giao tiếp 1-Wire (truyền tín hiệu số trên một dây duy nhất). Dữ liệu xuất ra đã được số hóa kèm theo bit kiểm tra lỗi (checksum), giúp vi điều khiển đọc tín hiệu chính xác mà không cần xử lý nhiễu analog.



Hình 2 Cảm biến DHT11

- **Cảm biến cường độ ánh sáng (Quang trở - LDR):**
    - Là một điện trở phụ thuộc vào ánh sáng. Khi cường độ ánh sáng chiếu vào tăng, điện trở của LDR giảm xuống và ngược lại.
    - Được mắc nối tiếp với một điện trở cố định tạo thành cầu phân áp. Sự thay đổi điện trở sẽ làm thay đổi điện áp tại điểm giữa, truyền vào chân analog của ESP8266 (chân A0) để nội suy ra mức độ ánh sáng.



Hình 3 Quang trở

- **Cơ cấu chấp hành (Module Relay 5V):**
    - Hoạt động như một công tắc điện từ, sử dụng dòng điện áp thấp (3.3V - 5V) từ vi điều khiển để kích hoạt cuộn dây, từ đó hút tiếp điểm đóng/ngắt mạch điện công suất lớn (như bóng đèn 220V).
    - Được tích hợp IC cách ly quang (Optocoupler), giúp bảo vệ vi mạch ESP8266 khỏi các hiện tượng sốc điện hay dòng ngược từ thiết bị điện lưới.

2.1.2 Lớp mạng và Giao thức truyền thông (Network & Protocol Layer)

Hệ thống sử dụng song song hai giao thức phổ biến nhất hiện nay để tối ưu hóa cho từng loại luồng dữ liệu khác nhau.

- **Giao thức MQTT (Message Queuing Telemetry Transport):**
    - Áp dụng cho luồng dữ liệu thời gian thực (giữa Phần cứng và Backend).
    - Hoạt động theo mô hình **Publish/Subscribe** (Xuất bản / Đăng ký) thông qua một máy chủ trung gian gọi là MQTT Broker.

- **Ưu điểm:** Khối lượng gói tin cực nhẹ (overhead chỉ 2 bytes), băng thông thấp, tối ưu cho mạng chập chờn. Có cơ chế QoS (Quality of Service) đảm bảo thông điệp điều khiển thiết bị không bị thất lạc.
- **Cấu trúc Topic (Chủ đề):** Phân chia rõ ràng như home/livingroom/sensors (để gửi data) và home/livingroom/light/cmd (để nhận lệnh).

- **Giao thức HTTP/RESTful API:**
  - Áp dụng cho luồng dữ liệu đồng bộ (giữa Frontend và Backend).
  - Được sử dụng khi người dùng truy vấn dữ liệu lịch sử (Data Sensor, Action History), đăng nhập, hoặc cập nhật Profile. Hoạt động theo mô hình Request/Response tiêu chuẩn.

2.1.3 Lớp Xử lý và Lưu trữ (Backend & Database Layer)

Đây là "bộ não" tính toán, phân luồng và lưu trữ dài hạn của hệ thống.

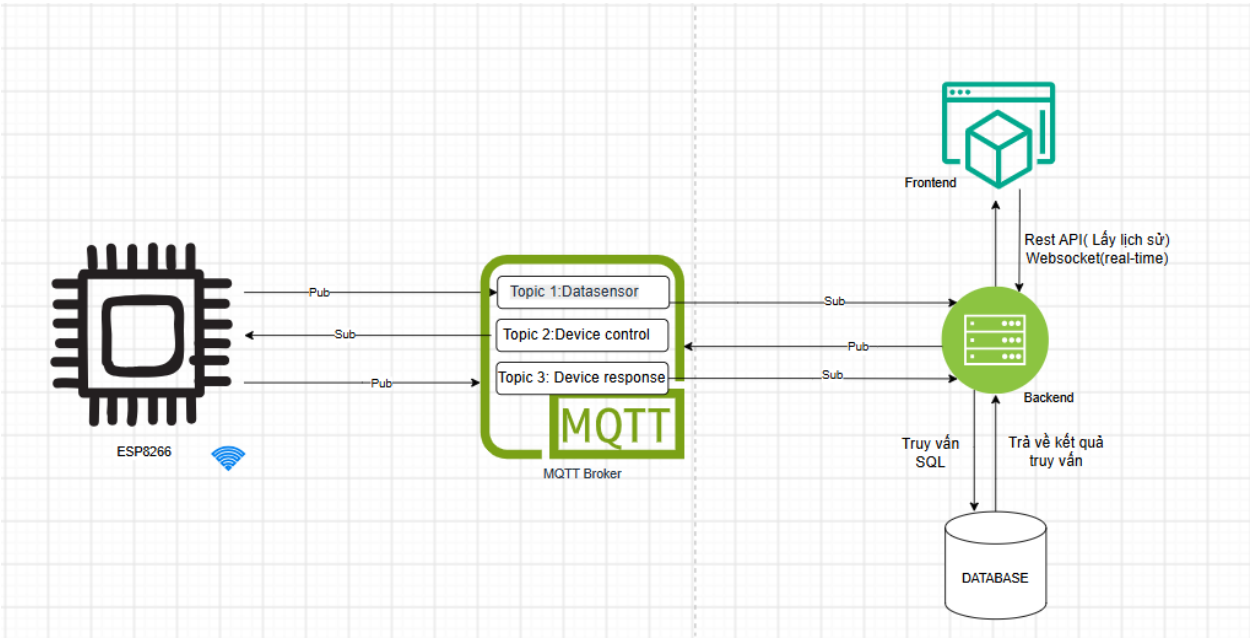
- **Máy chủ Backend (Node.js Runtime v26.x):**
  - Xây dựng hệ thống bằng ngôn ngữ Java, quản lý các thư viện và tiến trình đóng gói dự án thông qua Maven.
  - Mã nguồn được tổ chức theo mô hình MVC/3-Layer (Controller, Service, Repository), giúp tách biệt rõ ràng giữa logic xử lý nghiệp vụ, quản lý API, và giao tiếp cơ sở dữ liệu.
  - Tích hợp thư viện MQTT Client (như Eclipse Paho) để liên tục lắng nghe (subscribe) dữ liệu từ thiết bị, đồng thời mở các endpoint RESTful cung cấp dữ liệu cho Frontend.

- **Cơ sở dữ liệu Quan hệ (Hệ quản trị MySQL 10.4 (MariaDB qua XAMPP)):**
  - Lưu trữ các bảng dữ liệu cấu trúc khắt khe (người dùng, thiết bị, lịch sử cảm biến, lịch sử hành động).
  - Chuẩn hóa dữ liệu (Normalization): Lược đồ cơ sở dữ liệu được phân tích kỹ lưỡng các phụ thuộc hàm (Functional Dependencies) để đảm bảo không có phụ thuộc hàm sai lệch nào làm vô hiệu hóa khóa giả định (Candidate Key) ban đầu. Dữ liệu được đưa về ít nhất là dạng chuẩn 3 (3NF) hoặc BCNF, triệt tiêu hoàn toàn sự dư thừa và dị thường cập nhật.
  - Sử dụng Indexing trên các trường time để tối ưu hóa tốc độ truy vấn khi xuất báo cáo theo giây, phút, ngày.

2.1.4 Lớp Ứng dụng và Giao diện (Application / UI Layer)

Lớp này tương tác trực tiếp với người dùng cuối, được thiết kế dưới dạng bảng điều khiển (Dashboard) quản trị.

- Công cụ/ngôn ngữ sử dụng: HTML5, CSS3 thuần (CSS Grid, Flexbox),JavaScript thuần (Vanilla JS ES6+),Thư viện đồ thị Chart.js (v4.4.7)
- Hiển thị Thời gian thực (Real-time Rendering): Áp dụng công nghệ WebSocket hoặc Server-Sent Events (SSE). Ngay khi Backend nhận được dữ liệu cảm biến mới từ MQTT, nó sẽ đẩy (push) trực tiếp tín hiệu xuống trình duyệt. Giao diện sẽ tự động vẽ lại biểu đồ đa trục (Multi-axis Chart) mà không yêu cầu người dùng phải tải lại trang (Reload).
- Quản lý trạng thái (State Management): Các nút công tắc điều khiển thiết bị trên UI (Device Control) được xử lý logic bất đồng bộ (Asynchronous). Khi bấm bật/tắt, giao diện sẽ rơi vào trạng thái "Pending" chờ phản hồi xác nhận từ phần cứng gửi về thông qua API, đảm bảo tính toàn vẹn và chính xác tuyệt đối của lệnh.



Hình 4 Hệ thống trao đổi giữa phần cứng-lớp mạng-ứng dụng&giao diện

2.2 Cơ sở dữ liệu

+ Bảng Users (Quản lý tài khoản)

Tên cột	Kiểu dữ liệu	Ràng buộc (Constraints)
id	INT	**PK**, AUTO_INCREMENT
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL

+ Bảng Sensor (Danh mục cảm biến - MỚI)

Tên cột	Kiểu dữ liệu	Ràng buộc (Constraints)
sensor_id	VARCHAR(50)	**PK**
device_id	VARCHAR(50)	**FK → devices(device_id)**
name	VARCHAR(100)	NOT NULL
type	VARCHAR(50)	NOT NULL
unit(đơn vị)	VARCHAR(20)	NOT NULL

+Bảng devices (Danh mục thiết bị - Cập nhật trạng thái cuối)

Tên cột	Kiểu dữ liệu	Ràng buộc (Constraints)
device_id	VARCHAR(50)	**PK**
name	VARCHAR(100)	NOT NULL
room	VARCHAR(50)	NOT NULL
last_state	VARCHAR(20)	DEFAULT 'OFF'

+ Bảng action_history (Lịch sử điều khiển)

Tên cột	Kiểu dữ liệu	Ràng buộc (Constraints)
id	INT	**PK**, AUTO_INCREMENT
device_id	VARCHAR(50)	**FK → devices(device_id)**
action	VARCHAR(20)	NOT NULL
status	VARCHAR(20)	NOT NULL
time	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

+ Bảng datasensor (Dữ liệu cảm biến)

Tên cột	Kiểu dữ liệu	Ràng buộc (Constraints)
id	INT	**PK**, AUTO_INCREMENT
sensor_id	VARCHAR(50)	**FK → sensors(sensor_id)**
sensor_type	VARCHAR(50)	NOT NULL
sensor_value	FLOAT	NOT NULL

time	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

-Mối liên hệ

+ Sensor-Datasensor:1_N + Device-Sensor:1_N

+ Device-Action history:1_N

CHƯƠNG 3. THIẾT KẾ CHI TIẾT

3.1 *Đặc tả usecase*

3.1.1 Use case: Đăng nhập hệ thống(Login)

Mô tả: Người dùng nhập thông tin định danh (Email và Mật khẩu) để xác thực danh tính. Nếu hợp lệ, hệ thống cấp một chuỗi Token để phân quyền truy cập vào các trang Dashboard, Datasensor, v.v.

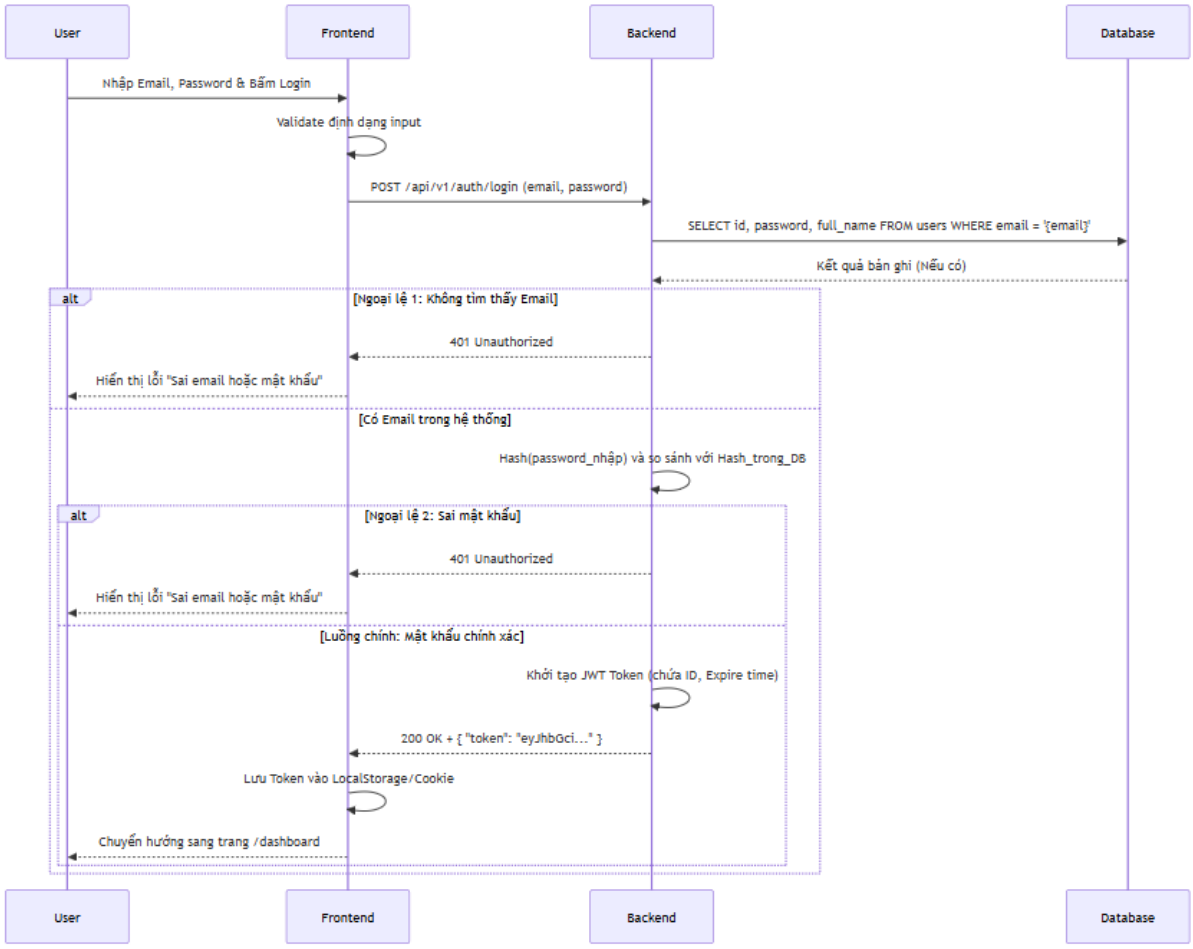
Luồng chính :

1. Người dùng nhập Email và Mật khẩu trên giao diện FE, bấm "Đăng nhập".
2. FE validate form cơ bản (không để trống, đúng định dạng email) và gửi HTTP POST request xuống BE.
3. BE tiếp nhận, thực hiện truy vấn DB để lấy thông tin tài khoản: SELECT id, password, full_name FROM users WHERE email = '{email}'.
4. BE sử dụng thuật toán băm (ví dụ: BCrypt hoặc SHA) để băm mật khẩu người dùng vừa nhập và so khớp với chuỗi mã băm (password) lấy từ DB.
5. So khớp thành công, BE tạo một chuỗi JWT (JSON Web Token) chứa user_id và thời gian hết hạn (Expiration).
6. BE trả về mã 200 OK kèm theo Token.
7. FE lưu Token vào Local Storage (hoặc Cookie) và chuyển hướng (Redirect) người dùng vào trang Dashboard.

Luồng ngoại lệ :

- **Ngoại lệ 1 - Sai thông tin:** DB trả về rỗng (không tìm thấy Email) hoặc quá trình so khớp mã băm mật khẩu thất bại. BE trả về mã lỗi 401 Unauthorized. FE hiển thị thông báo chung: *"Sai email hoặc mật khẩu"* (Lưu ý: Không thông báo chi tiết là sai email hay sai mật khẩu để tránh kẻ gian dò quét tài khoản).
- **Ngoại lệ 2 - Chống tấn công dò mật khẩu:** Nếu người dùng nhập sai quá 5 lần liên tiếp, BE ghi nhận tần suất và trả về lỗi 429 Too Many Requests, tạm khóa đăng

nhập trong 15 phút. FE hiển thị cảnh báo "Tài khoản tạm khóa do nhập sai nhiều lần".



Hình 5 Biểu đồ sequence cho đăng nhập hệ thống

Mô tả sequence:

Bước 1: Người dùng nhập thông tin định danh gồm Email và Mật khẩu trên giao diện Frontend (FE), sau đó bấm "Đăng nhập".

Bước 2: FE tiến hành kiểm tra (validate) form cơ bản đảm bảo không để trống, đúng định dạng email và gửi một HTTP POST request chứa dữ liệu xuống Backend (BE).

Bước 3: BE tiếp nhận request và thực thi câu lệnh truy vấn SQL vào cơ sở dữ liệu (Database) để lấy thông tin tài khoản: SELECT id, password, full_name FROM users WHERE email = '{email}'.

Bước 4: BE sử dụng thuật toán băm (ví dụ: BCrypt hoặc SHA) để mã hóa mật khẩu người dùng vừa nhập, sau đó so khớp với chuỗi mã băm (password) đã được lấy từ Database.

Ngoại lệ

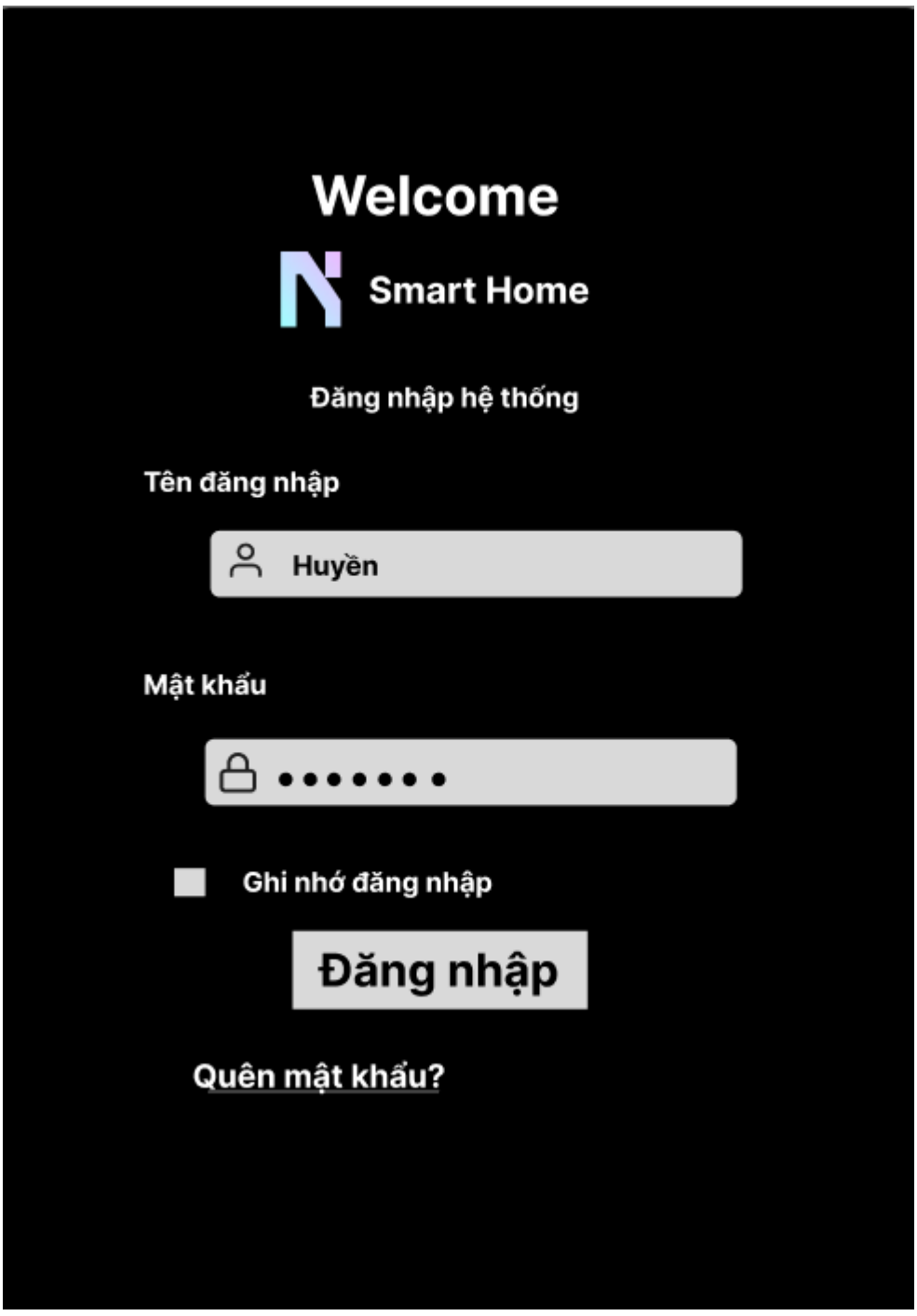
Bước 5 (Ngoại lệ): Nếu thông tin không khớp hoặc DB trả về rỗng (không tìm thấy Email), BE trả về mã lỗi 401 Unauthorized và FE hiển thị thông báo "Sai email hoặc mật khẩu".

Bước 6 (Ngoại lệ): Nếu người dùng nhập sai quá 5 lần liên tiếp, BE ghi nhận tần suất, trả về lỗi 429 Too Many Requests và FE hiển thị cảnh báo "Tài khoản tạm khóa do nhập sai nhiều lần".

Bước 7: Nếu so khớp thành công, BE tạo một chuỗi JWT (JSON Web Token) chứa user_id và thời gian hết hạn (Expiration), sau đó trả kết quả về cho FE với mã 200 OK kèm theo Token.

Bước 8: FE nhận Token, lưu trữ vào Local Storage (hoặc Cookie) và thực hiện chuyển hướng (Redirect) người dùng vào trang Dashboard.

+ Giao diện Login



Hình 6 Giao diện Login

3.1.2 Use case: Giám sát dữ liệu thời gian thực (Dashboard)

Mô tả: Người dùng xem các thông số nhiệt độ, độ ẩm, ánh sáng mới nhất và biểu đồ biến thiên trên màn hình chính.

Tiền điều kiện:Đăng nhập thành công vào hệ thống

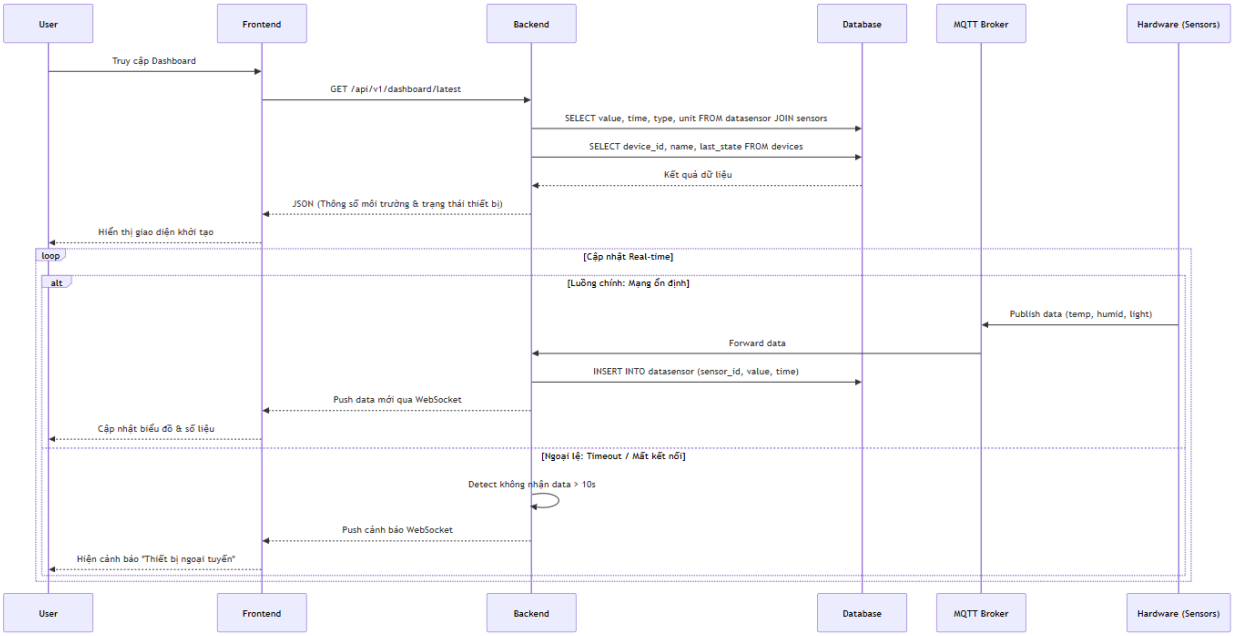
Hậu điều kiện:Xem được nhiệt độ, độ ẩm, ánh sáng

- **Luồng chính:**

1. Người dùng truy cập trang Dashboard.

2. FE gọi API lấy dữ liệu hiện tại từ BE.
3. BE truy vấn DB lấy bản ghi mới nhất: Lấy trường value, time từ bảng datasensor (JOIN với bảng sensors qua trường sensor_id để lấy type và unit) với điều kiện thời gian gần nhất (ORDER BY time DESC LIMIT 1 cho mỗi loại) và trả về FE hiển thị.
4. Hardware đọc cảm biến liên tục, gửi data lên MQTT, thực hiện lệnh INSERT các trường (sensor_id, value, time) vào bảng datasensor và push qua WebSocket cho FE.
5. BE nhận data từ MQTT, lưu vào DB và đẩy dữ liệu mới xuống FE (qua WebSocket/SSE) để cập nhật giao diện mà không cần reload.

- **Luồng ngoại lệ:**
  - *Mất kết nối MQTT/Hardware offline:* BE không nhận được dữ liệu mới quá thời gian cho phép (timeout), FE hiển thị cảnh báo "Thiết bị ngoại tuyến".



Hình 7 Biểu đồ sequence của usecase Giám sát dữ liệu thời gian thực

Mô tả sequence :

- Bước 1: Người dùng mở trình duyệt và truy cập vào trang Dashboard của hệ thống.
- Bước 2: Frontend (FE) tự động gọi API (thông qua giao thức HTTP GET) gửi đến Backend (BE) để yêu cầu lấy các thông số cảm biến và trạng thái thiết bị hiện tại.
- Bước 3: BE tiếp nhận yêu cầu và thực thi câu lệnh truy vấn SQL vào cơ sở dữ liệu (Database). Cụ thể, BE thực hiện lệnh JOIN bảng datasensor với bảng sensors

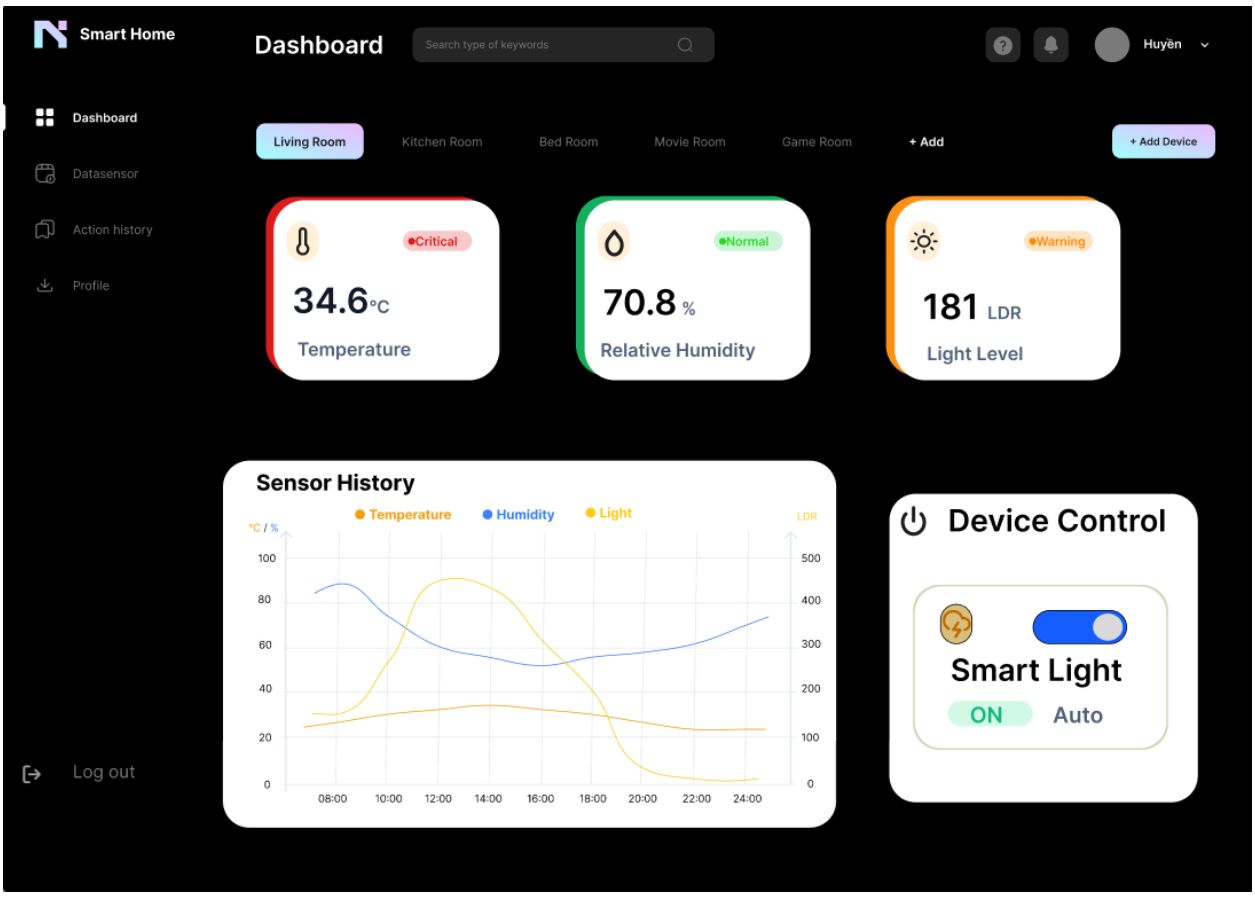
(thông qua khóa sensor_id) để lấy đơn vị (unit) và loại (type), đồng thời dùng ORDER BY time DESC LIMIT 1 để rút trích ra bản ghi có thời gian mới nhất cho từng loại cảm biến.

- Bước 4: Database trả kết quả lại cho BE. BE đóng gói dữ liệu này thành định dạng JSON và phản hồi về cho FE. FE sử dụng dữ liệu này để vẽ biểu đồ Sensor History và hiển thị các thông số (nhiệt độ, độ ẩm, ánh sáng) lần đầu tiên lên màn hình.
- Bước 5: Ở môi trường vật lý, vi điều khiển trung tâm (ESP8266) liên tục đọc tín hiệu từ các cảm biến (DHT11 đo nhiệt độ/độ ẩm, LDR đo cường độ ánh sáng).
- Bước 6: ESP8266 đóng gói các giá trị vừa đọc được và liên tục xuất bản (Publish) luồng dữ liệu này lên trung tâm điều phối MQTT Broker thông qua một Topic đã định nghĩa (ví dụ: datasensor).
- Bước 7: BE (luôn chạy ngầm một tiến trình MQTT Client để lắng nghe - Subscribe) ngay lập tức nhận được gói dữ liệu từ MQTT Broker.
- Bước 8: Nhận được data, BE thực hiện lệnh INSERT INTO để lưu các trường sensor_id, value, time vào bảng datasensor trong cơ sở dữ liệu nhằm phục vụ việc tra cứu lịch sử sau này.
- Bước 9: Đồng thời với việc lưu Database, BE sử dụng kết nối WebSocket (hoặc Server-Sent Events - SSE) đang mở sẵn với trình duyệt để đẩy (Push) trực tiếp dữ liệu mới này thẳng xuống FE.
- Bước 10: FE tiếp nhận dữ liệu từ WebSocket, kích hoạt hệ thống quản lý trạng thái (như State của ReactJS) để tự động dịch chuyển điểm neo trên biểu đồ và nhảy số liệu mới mà không cần người dùng phải nhấn F5 (reload) trang web.

Luồng ngoại lệ: Xử lý khi mất kết nối phần cứng

- Bước 1 (Ngoại lệ): Vì một lý do vật lý (mất điện, rớt mạng Wi-Fi, hỏng cảm biến), ESP8266 ngừng gửi các bản tin (payload) lên MQTT Broker.
- Bước 2 (Ngoại lệ): Bộ đếm thời gian (Timer) trên BE phát hiện hệ thống đã vượt quá ngưỡng thời gian chờ cho phép (Timeout, ví dụ: 10 giây) mà không nhận được bất kỳ tín hiệu cập nhật nào từ MQTT.
- Bước 3 (Ngoại lệ): BE kích hoạt logic báo lỗi, đẩy một thông điệp cảnh báo mất kết nối qua kênh WebSocket xuống cho FE.
- Bước 4 (Ngoại lệ): FE nhận thông điệp cảnh báo, lập tức cập nhật giao diện, đổi màu các thẻ thông số sang trạng thái ngắt kết nối và hiển thị nhãn cảnh báo "Thiết bị ngoại tuyến" để người dùng kịp thời nắm bắt sự cố.

+ Giao diện Dashboard



Hình 8 Giao diện Dashboard

3.1.3 Use case: Tra cứu và lọc dữ liệu cảm biến (Data Sensor)

Mô tả: Người dùng tìm kiếm lịch sử dữ liệu cảm biến (id, loại, giá trị, time) theo khoảng thời gian cụ thể (giây, phút, giờ, ngày).

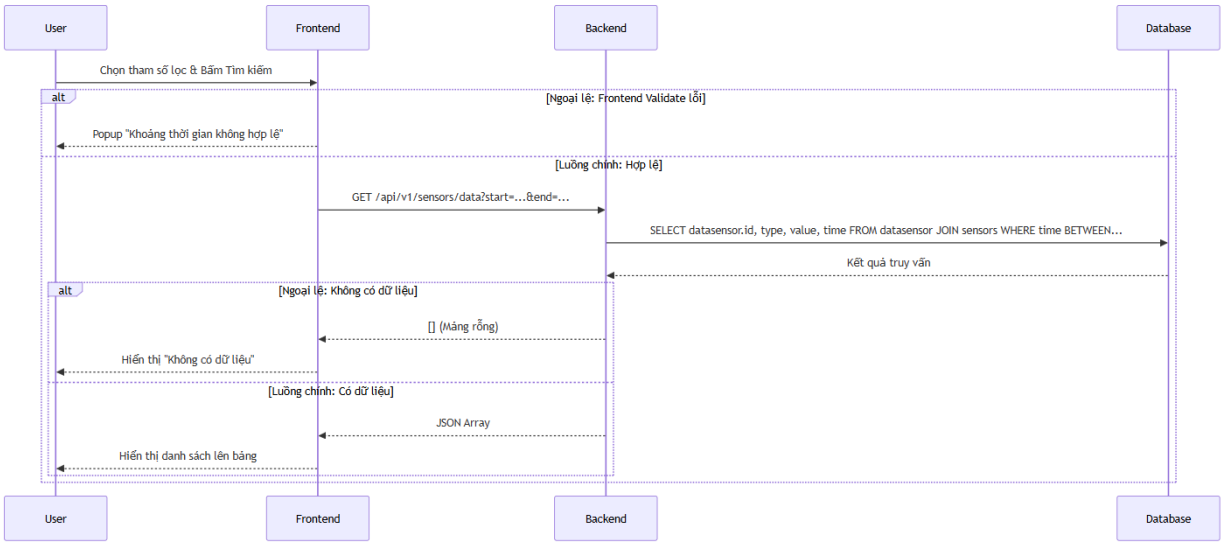
Tiền điều kiện:Đăng nhập thành công

Hậu điều kiện:Hiển thị bảng giá trị (id, loại, giá trị, time) theo khoảng thời gian cụ thể (giây, phút, giờ, ngày)

- **Luồng chính:**

1. Người dùng chọn loại cảm biến và thời gian bắt đầu/kết thúc trên giao diện Data Sensor.
2. Người dùng bấm "Tìm kiếm".
3. FE gửi request chứa các tham số lọc xuống BE.
4. BE query đến các trường các trường datasensor.id, sensors.type, datasensor.value, datasensor.time trong Database để lấy danh sách dữ liệu phù hợp.
5. BE trả kết quả về FE để hiển thị dưới dạng bảng.

- **Luồng ngoại lệ:**
    - *Không có dữ liệu:* DB trả về mảng rỗng, FE hiển thị thông báo "Không tìm thấy dữ liệu trong khoảng thời gian này".
    - *Lỗi tham số thời gian:* Người dùng nhập Thời gian bắt đầu > Thời gian kết thúc, FE chặn ngay tại client hoặc BE trả về lỗi 400 Bad Request.



Hình 9 Biểu đồ sequence tra cứu và lọc dữ liệu cảm biến

Mô tả:

Bước 1: Người dùng thao tác trên giao diện trang Data Sensor, sử dụng các hộp thoại chọn (dropdown) để chỉ định loại cảm biến cần xem và sử dụng công cụ chọn thời gian để thiết lập mốc thời gian bắt đầu cùng thời gian kết thúc.

Bước 2: Người dùng xác nhận điều kiện và bấm vào nút "Tìm kiếm" trên giao diện.

Bước 3 (Luồng ngoại lệ): FE tiến hành kiểm tra logic thời gian ngay tại trình duyệt. Nếu thời gian bắt đầu lớn hơn thời gian kết thúc, FE sẽ chặn yêu cầu gọi API (hoặc BE sẽ bắt lỗi này và trả về mã lỗi HTTP 400 Bad Request).

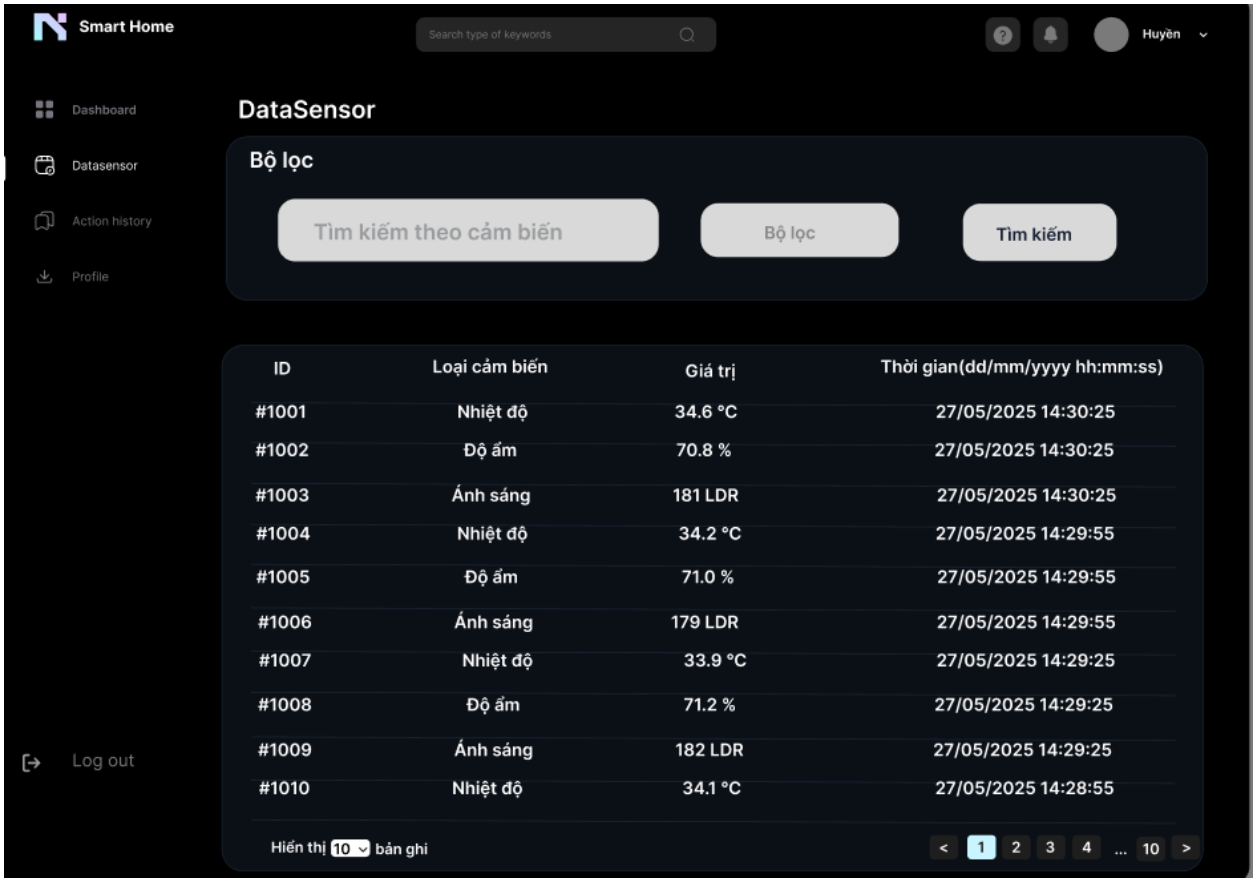
Bước 4: FE đóng gói các tham số bộ lọc vào cấu trúc Query String (?type={...}&start={...}&end={...}) và gửi một HTTP GET request (tới endpoint /api/v1/sensors/data) xuống BE.

Bước 5: BE tiếp nhận tham số từ request và tiến hành xây dựng câu lệnh truy vấn SQL động. BE thực hiện lệnh SELECT đến các trường datasensor.id, sensors.type, datasensor.value, datasensor.time trong Database, kết hợp với lệnh JOIN giữa bảng datasensor và sensors thông qua khóa ngoại để lấy danh sách dữ liệu phù hợp với khoảng thời gian yêu cầu.

Bước 6 (Luồng ngoại lệ): Nếu câu lệnh truy vấn Database không tìm thấy bản ghi nào khớp với điều kiện (DB trả về mảng rỗng), BE trả kết quả này cho FE, và FE sẽ hiển thị một thông báo cảnh báo "Không tìm thấy dữ liệu trong khoảng thời gian này".

Bước 7: Nếu có dữ liệu hợp lệ, Database trả kết quả lại cho BE, BE đóng gói thành định dạng JSON Array và phản hồi cho FE. FE sử dụng mảng dữ liệu này để render trực tiếp thành các hàng trong bảng dữ liệu hiển thị cho người dùng.

+Giao diện Datasensor



Hình 10 Giao diện Datasensor

3.1.4 Use case:Device control(Điều khiển thiết bị)

Mô tả: Người dùng thực hiện thao tác bật/tắt các thiết bị ngoại vi (đèn) trực tiếp từ Dashboard. Thao tác này sinh ra lịch sử.

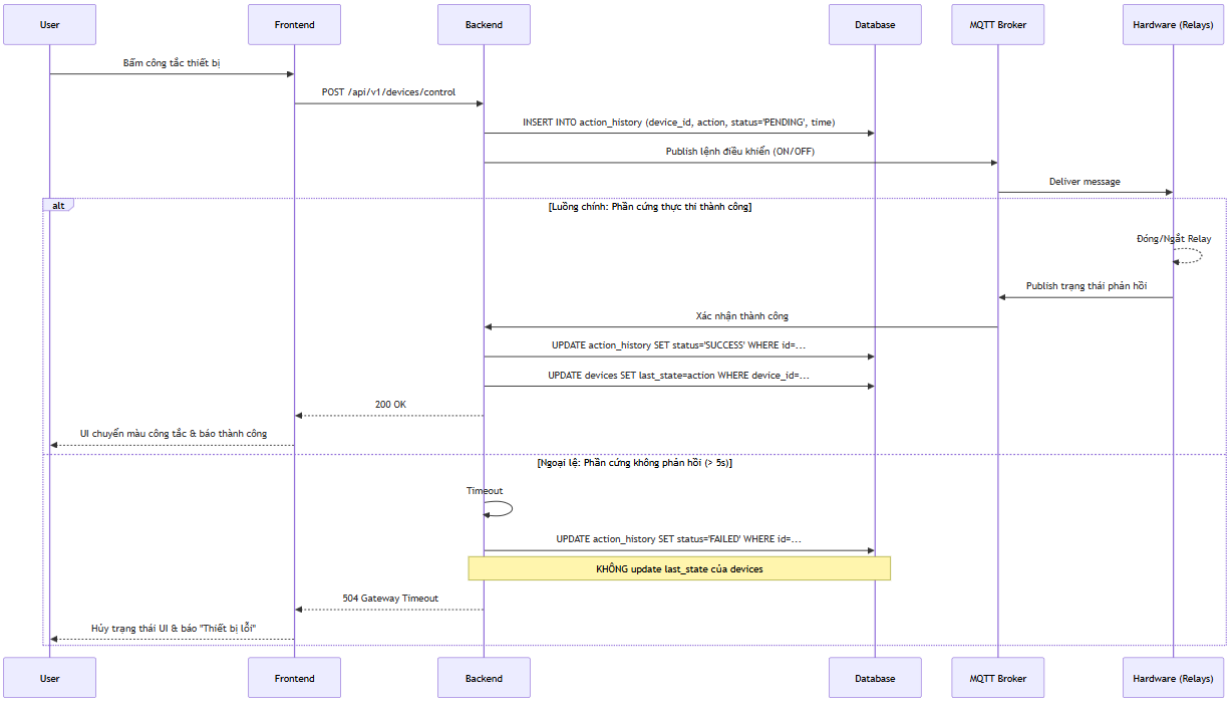
Tiền điều kiện: Đăng nhập hệ thống

Hậu điều kiện: Bật/tắt thiết bị thành công

- **Luồng chính:**
    1. Người dùng bấm nút Bật/Tắt thiết bị trên FE.
    2. FE gọi API điều khiển xuống BE.
    3. BE thực hiện lệnh INSERT vào bảng action_history các trường (device_id, action, status = 'PENDING', time).
    4. BE publish lệnh điều khiển lên MQTT broker.

5. Hardware (ESP8266) nhận lệnh, thực thi đóng/cắt Relay và publish trạng thái xác nhận lại MQTT.
6. Khi phần cứng báo thành công, BE thực hiện 2 câu lệnh DB:
    - UPDATE trường status = 'SUCCESS' trong bảng action_history dựa theo id giao dịch.
    - UPDATE trường last_state = giá trị của action trong bảng devices dựa theo device_id.
7. BE nhận xác nhận, update DB thành SUCCESS và phản hồi cho FE.

- **Luồng ngoại lệ:**
    - *Phần cứng lỗi:* Hardware không phản hồi sau 5 giây, BE cập nhật trạng thái trong DB thành FAILED và trả lỗi 504 Gateway Timeout về FE. FE báo "Điều khiển thất bại".



Hình 11 Biểu đồ sequence điều khiển thiết bị

Mô tả sequence:

Bước 1: Tại khu vực thẻ "Device Control" trên giao diện Dashboard, người dùng thực hiện thao tác bấm vào nút công tắc Bật/Tắt đối với thiết bị ngoại vi (ví dụ: đèn).

Bước 2: Giao diện chuyển nút bấm sang dạng chờ (Pending), đồng thời FE đóng gói thông tin thiết bị và lệnh điều khiển, sau đó gửi HTTP POST request (tới endpoint /api/v1/devices/control) chứa tham số xuống BE.

Bước 3: BE tiếp nhận luồng dữ liệu, tiến hành bóc tách và thực thi ngay câu lệnh SQL INSERT INTO vào bảng action_history. BE sẽ khởi tạo luồng lịch sử bằng cách lưu các

trường (device_id, action, time) với trạng thái ép cứng mặc định là status = 'PENDING' nhằm đánh dấu lệnh đang chờ xử lý.

Bước 4: Sau khi ghi nhận trên cơ sở dữ liệu, tiến trình bên trong BE sẽ xuất bản (Publish) chuỗi lệnh điều khiển Bật/Tắt này lên hệ thống MQTT broker.

Bước 5: Thiết bị phần cứng (ESP8266) thông qua kết nối Wi-Fi sẽ nhận được lệnh điều khiển từ broker, tiến hành thực thi việc đóng hoặc cắt module Relay 5V, sau đó publish ngược lại thông điệp trạng thái xác nhận lên mạng MQTT.

Bước 6 (Luồng ngoại lệ): Nếu qua 5 giây kể từ lúc BE gửi lệnh mà không nhận được bất kỳ tín hiệu xác nhận nào từ thiết bị (Timeout), BE sẽ tự động gọi truy vấn UPDATE bảng action_history đổi trạng thái thành FAILED, đồng thời KHÔNG tiến hành cập nhật trạng thái last_state của bảng thiết bị, và trả mã lỗi HTTP 504 Gateway Timeout về FE để FE hủy giao diện đang chờ và báo "Điều khiển thất bại".

Bước 7: Khi phần cứng thực thi thành công và báo cáo xác nhận về hệ thống MQTT, BE tiếp nhận thông điệp, đối chiếu giao dịch và thực thi đồng thời hai câu lệnh SQL: UPDATE trường status = 'SUCCESS' cho bản ghi giao dịch trong bảng action_history, và UPDATE trường last_state của thiết bị tương ứng bằng chính giá trị action trong bảng devices.

Bước 8: Hoàn tất việc cập nhật Database, BE phản hồi mã trạng thái HTTP 200 OK về FE. FE xử lý luồng dữ liệu này bằng cách đổi màu sắc của công tắc trên UI và hiển thị thông báo thao tác đã thành công cho người dùng.

3.1.5 Use case: Xem lịch sử hoạt động (Action History)

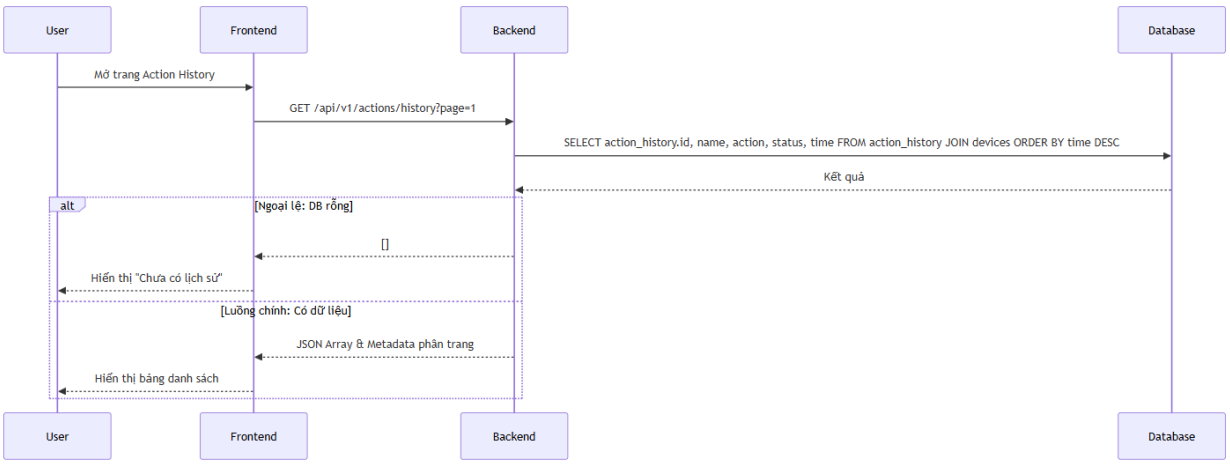
Mô tả: Người dùng tra cứu danh sách các lệnh đã điều khiển (id, thiết bị, action, status, time).

Tiền điều kiện: Đăng nhập thành công

Hậu điều kiện: Xem dữ liệu đã điều khiển (id, thiết bị, action, status, time).

- **Luồng chính:**
    1. Người dùng chuyển sang tab Action History.
    2. FE gọi API lấy danh sách lịch sử hành động.
    3. BE thực hiện SELECT các trường action_history.id, devices.name, action_history.action, action_history.status, action_history.time bằng cách JOIN hai bảng action_history và devices thông qua device_id
    4. Kết quả được sắp xếp ORDER BY time DESC và phân trang.
    5. FE hiển thị lên bảng.
- **Luồng ngoại lệ:**

- *Quá tải dữ liệu:* Nếu người dùng chọn hiển thị 10,000 bản ghi cùng lúc, BE có thể chặn lại, chỉ trả về tối đa 100 bản ghi/trang và yêu cầu FE hiển thị phân trang để giảm tải cho DB.



Hình 12 Biểu đồ sequence xem lịch sử hoạt động

Mô tả:

Bước 1: Người dùng thao tác nhấp chuột chuyển sang tab "Action History" để tra cứu thông tin vận hành trên thanh menu điều hướng của hệ thống.

Bước 2: Giao diện FE tự động khởi tạo luồng dữ liệu, gửi một HTTP GET request (tới endpoint /api/v1/actions/history) đính kèm theo các tham số phân trang tới máy chủ BE.

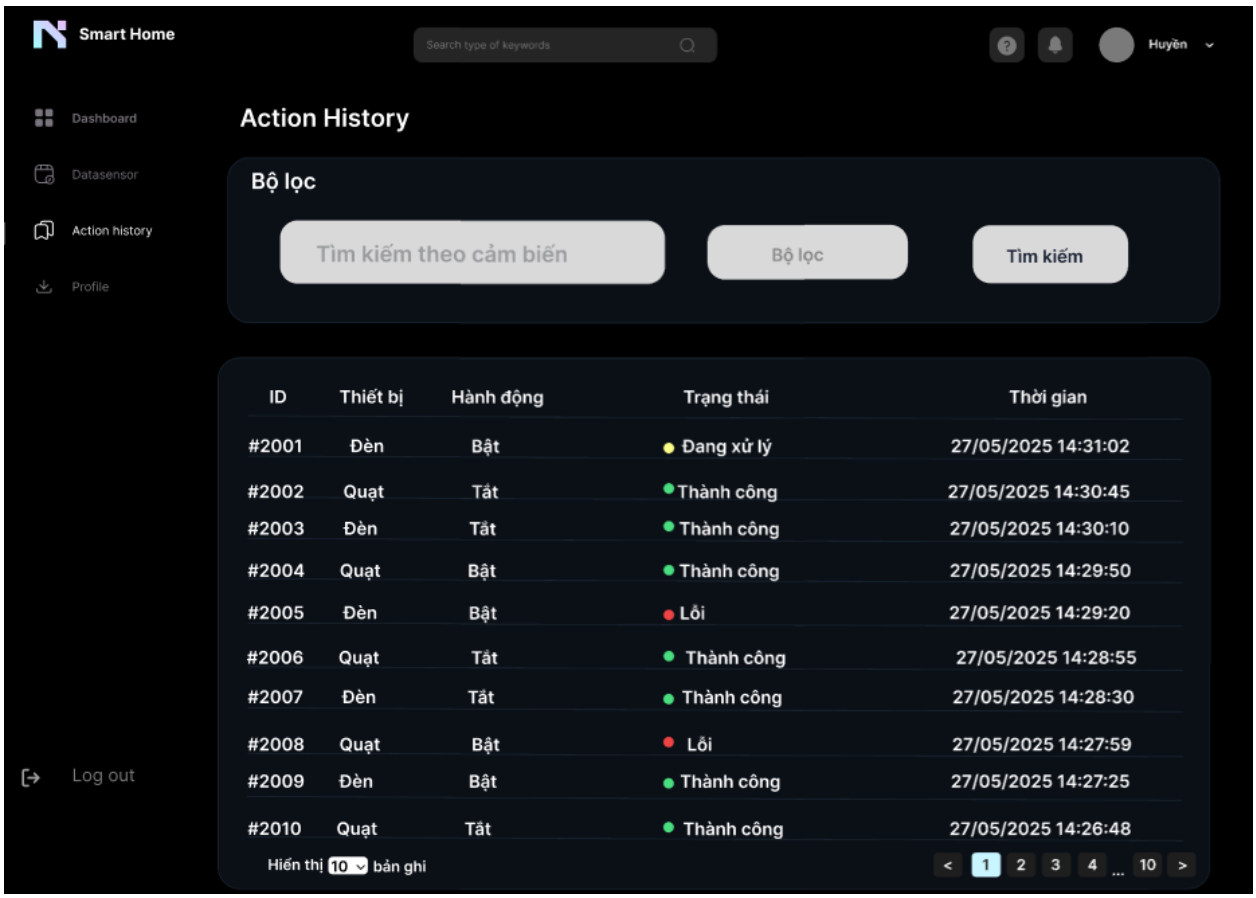
Bước 3: BE tiếp nhận yêu cầu lấy danh sách và thực hiện thiết lập câu truy vấn Database bằng lệnh SELECT để trích xuất các trường hiển thị gồm action_history.id, devices.name, action_history.action, action_history.status, và action_history.time.

gBước 4: Để thu thập đủ dữ liệu hiển thị (tên thiết bị), truy vấn của BE sẽ thực hiện nối bảng thông qua mệnh đề JOIN hai bảng action_history và devices bằng khóa liên kết device_id, sau đó kết quả thu thập sẽ được sắp xếp trật tự thời gian mới nhất lên đầu bằng mệnh đề ORDER BY time DESC.

Bước 5 (Luồng ngoại lệ): Nhằm tránh tình trạng quá tải, nếu truy vấn trả về một lượng dữ liệu khổng lồ (ví dụ: 10,000 bản ghi), BE sẽ chặn thao tác, chỉ trả về tối đa một số lượng bản ghi nhất định (ví dụ: 100 bản ghi/trang) và yêu cầu FE bắt buộc phải hiển thị phân trang để giảm tải truy xuất Database.

Bước 6: Cơ sở dữ liệu trả mảng dữ liệu đã phân trang cho BE, BE đóng gói thành cấu trúc JSON Arrays kèm theo metadata và phản hồi về FE. FE tiếp nhận thông tin và hiển thị danh sách chi tiết các lệnh điều khiển lên cấu trúc bảng UI.

+Giao diện Action History



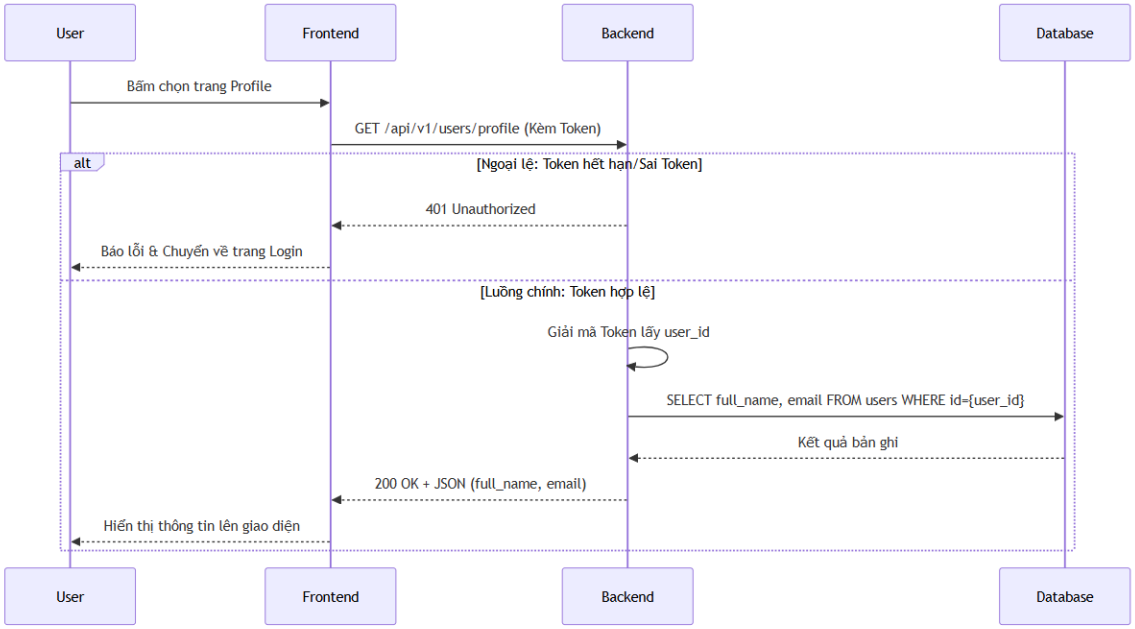
Hình 13 Giao diện Action History

3.1.6 Use case: Xem thông tin cá nhân (Profile)

- **Mô tả:** Người dùng xem các thông tin cá nhân (họ tên, email, v.v.) được trích xuất từ cơ sở dữ liệu dựa trên tài khoản đang đăng nhập.
- **Tiền điều kiện:** Người dùng đã đăng nhập thành công vào hệ thống, trình duyệt (Frontend) có lưu trữ Token xác thực hợp lệ.
- **Hậu điều kiện:** Hiển thị đúng và đầy đủ thông tin cá nhân của người dùng lên giao diện Profile.
- **Luồng chính:**
    1. Người dùng bấm chọn tab "Profile" trên thanh menu điều hướng.
    2. FE gọi API lấy thông tin người dùng (GET /api/v1/users/profile) kèm theo chuỗi Token xác thực trong HTTP Header.
    3. BE tiếp nhận request, tiến hành giải mã (verify) Token để trích xuất id của người dùng.
    4. BE thực hiện truy vấn cơ sở dữ liệu: SELECT full_name, email FROM users WHERE id = '{id}'.

5. BE đóng gói dữ liệu thành chuỗi JSON và trả kết quả về cho FE với mã trạng thái 200 OK.
6. FE nhận dữ liệu và hiển thị full_name, email lên các trường tương ứng trên giao diện Profile.

- **Luồng ngoại lệ:**
  - **Lỗi xác thực (Token hết hạn/Không hợp lệ):** Nếu Token đã quá thời gian sử dụng hoặc bị sai lệch, BE từ chối truy cập và trả về mã lỗi 401 Unauthorized. FE nhận lỗi, hiển thị thông báo "Phiên đăng nhập đã hết hạn", tiến hành xóa Token cũ ở Local Storage và tự động chuyển hướng (redirect) người dùng về trang Login.



Hình 14 Biểu đồ sequence xem thông tin cá nhân

Mô tả sequence :

Bước 1: Người dùng nhấn chọn vào mục "Profile" trên giao diện thanh điều hướng để truy cập vào trang hồ sơ cá nhân của tài khoản.

Bước 2: Quá trình render thành phần UI diễn ra, FE sẽ trích xuất chuỗi Token xác thực đang lưu trữ cục bộ, đính kèm Token này vào HTTP Header và gọi một HTTP GET API (/api/v1/users/profile) gửi lên máy chủ BE.

Bước 3 (Luồng ngoại lệ): Khi gói tin tiếp cận hệ thống phân quyền của BE, nếu Token phát hiện đã hết thời gian sử dụng, không hợp lệ hoặc bị sai lệch thông tin, BE sẽ từ chối cung cấp luồng dữ liệu và lập tức trả về mã lỗi HTTP 401 Unauthorized.

Bước 4 (Luồng ngoại lệ): Khi FE nhận được mã lỗi 401, nó sẽ hiển thị popup cảnh báo "Phiên đăng nhập đã hết hạn", thực thi tiến trình xóa Token cũ đã hết hạn khỏi bộ nhớ Local Storage, và tự động điều hướng (redirect) bắt buộc người dùng trở lại trang đăng nhập.

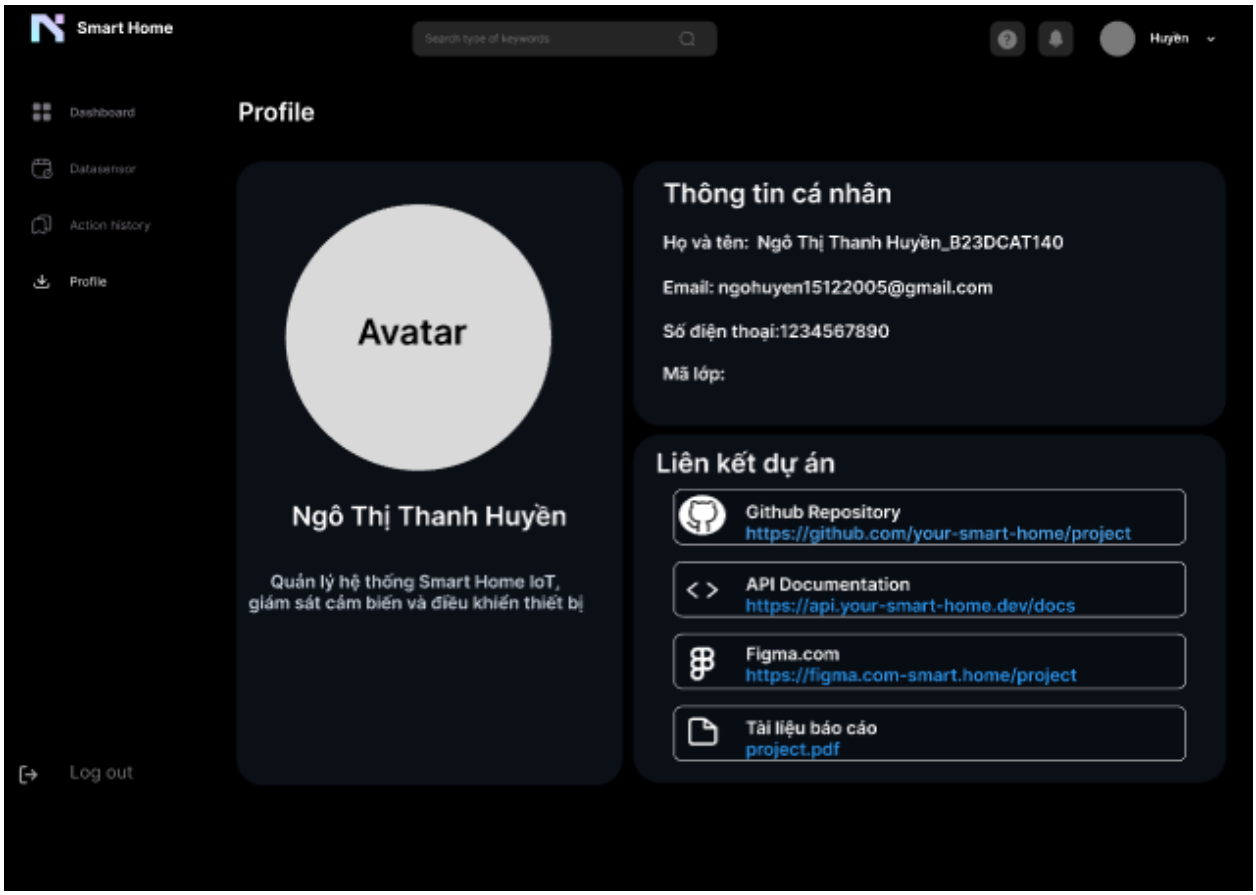
Bước 5: Nếu Token trong request gửi đi được hệ thống BE xác thực hoàn toàn hợp lệ, máy chủ sẽ tiến hành giải mã (verify) payload của đoạn mã JWT này để trích xuất trực tiếp thông số id của chính người dùng đang truy cập.

Bước 6: Mang thông số id này, BE gọi vào Database và thực thi một truy vấn độc lập: SELECT full_name, email FROM users WHERE id = '{id}' nhằm lấy ra định danh chính xác của tài khoản.

Bước 7: Cơ sở dữ liệu xử lý trả về kết quả, BE đóng gói các dữ liệu này thành luồng chuỗi định dạng JSON và đẩy kết quả về cho FE với mã trạng thái HTTP 200 OK.

Bước 8: Cuối cùng, FE nhận và bóc tách gói dữ liệu JSON thành công, xử lý đổ trực tiếp các thông số trích xuất (full_name, email) hiển thị lên các trường thông tin tương ứng ở bên trong thành phần hiển thị của giao diện Profile.

+Giao diện Profile



Hình 15 Giao diện Profile

3.2 API

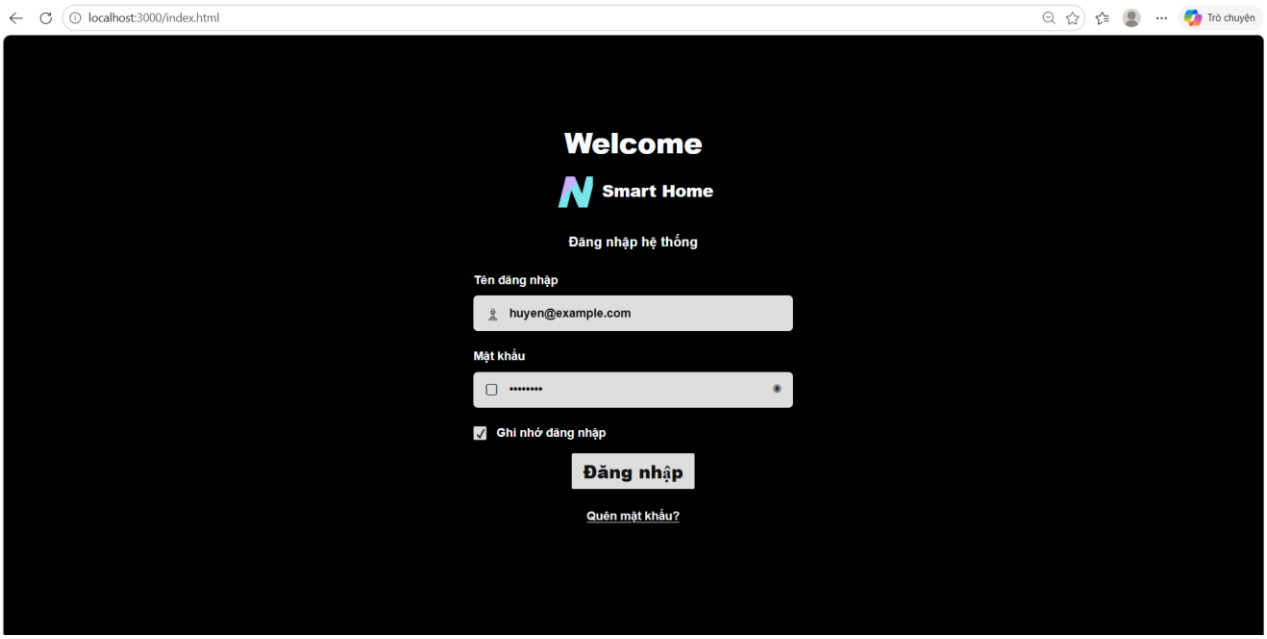
Use Case	HTTP Method	Cấu trúc Endpoint (URL)	Tham số (Query / Body)	Mục đích
Login	POST	/api/v1/auth/login	Body: {username,password}	Đăng nhập hệ thống
Dashboard	GET	/api/v1/dashboard/latest	Không cần tham số	Truy xuất bản ghi chứa 3 thông số môi trường gần nhất.
Data Sensor	GET	/api/v1/sensors/data	Query: ?type={...}&start={...}&end={...}	Lấy danh sách thông số theo dải thời gian để vẽ bảng.
Action Control	POST	/api/v1/devices/control	Body: { "deviceId": "...", "action": "ON" }	Phát lệnh bật/tắt thiết bị và lưu log trạng thái PENDING.
Action History	GET	/api/v1/actions/history	Query: ?page={...}&limit={...}	Lấy danh sách lịch sử thao tác thiết bị, có phân trang.
Profile	GET	/api/v1/users/profile	Header: Authorization: Bearer {Token}	Trích xuất thông tin cá nhân (Tên, Email) của người dùng đang đăng nhập dựa vào Token xác thực.

S

CHƯƠNG 4. KẾT QUẢ THỰC HIỆN

4.1 Chức năng Đăng nhập hệ thống (Authentication)

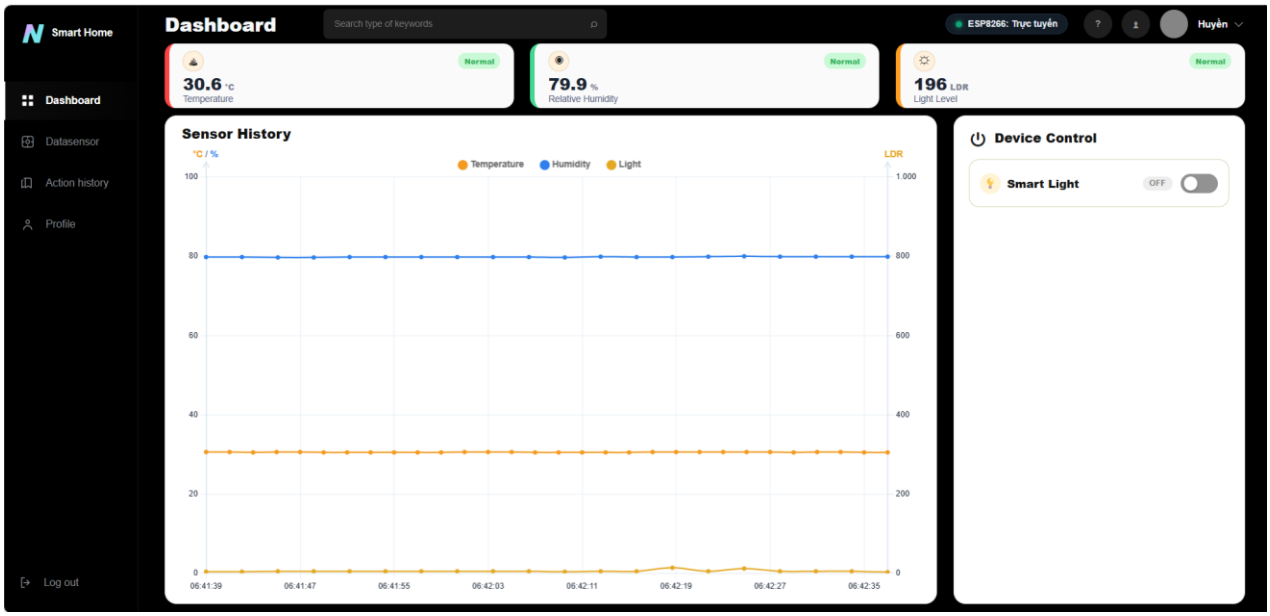
- **Kết quả:** Hệ thống cho phép đăng nhập linh hoạt bằng Email (ngohuyen15122005@gmail.com), Họ tên sinh viên hoặc Mã sinh viên (B23DCAT140).
- **Hoạt động:** Khi người dùng gửi form, Backend thực hiện truy vấn bảng users trong MySQL để xác thực. Khi thành công, hệ thống cấp phát token phiên làm việc và chuyển hướng tự động sang Dashboard.
- Sau khi ấn Đăng nhập, hệ thống hiển thị giao diện dashboard



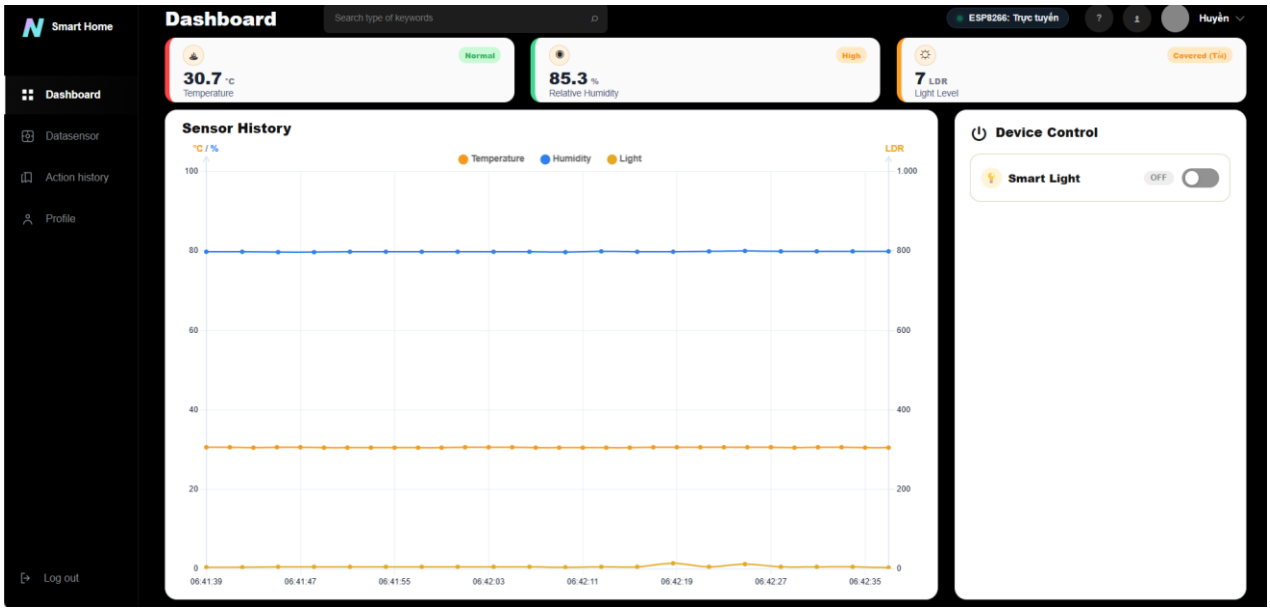
Hình 16 Giao diện đăng nhập hệ thống

4.2 Màn hình Dashboard giám sát thời gian thực

- **Kết quả:**
    - Hiển thị tức thời 03 thẻ thông số môi trường: Nhiệt độ (°C), Độ ẩm (%) và Ánh sáng (LDR).
    - Biểu đồ đường kép (Dual-Axis Chart) sử dụng Chart.js trực quan hóa lịch sử đo theo trục thời gian thực.
    - Khi che tay lên cảm biến quang trở (LDR), số đo ánh sáng lập tức giảm mạnh (dưới 150 LDR), thẻ trạng thái tự động chuyển sang nhãn cảnh báo Covered (Tối) và đồ thị cập nhật ngay lập tức.



Hình 17 Giao diện Dashboard hiển thị số đo thực tế và biểu đồ

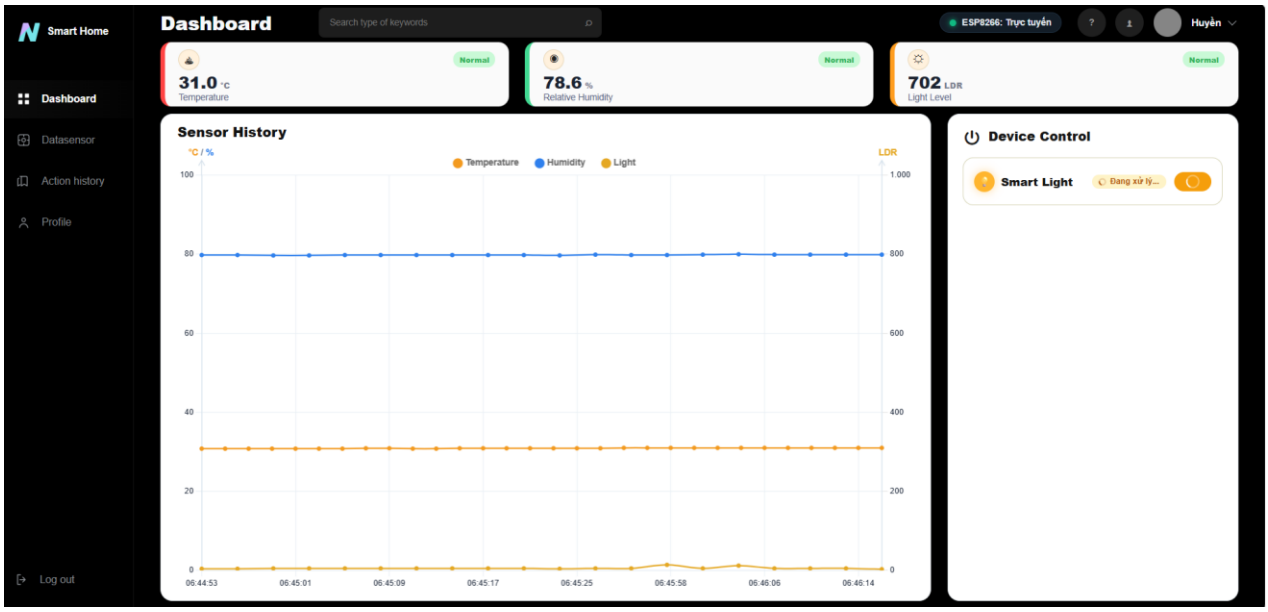


Hình 18 Trạng thái khi che cảm biến quang trở])

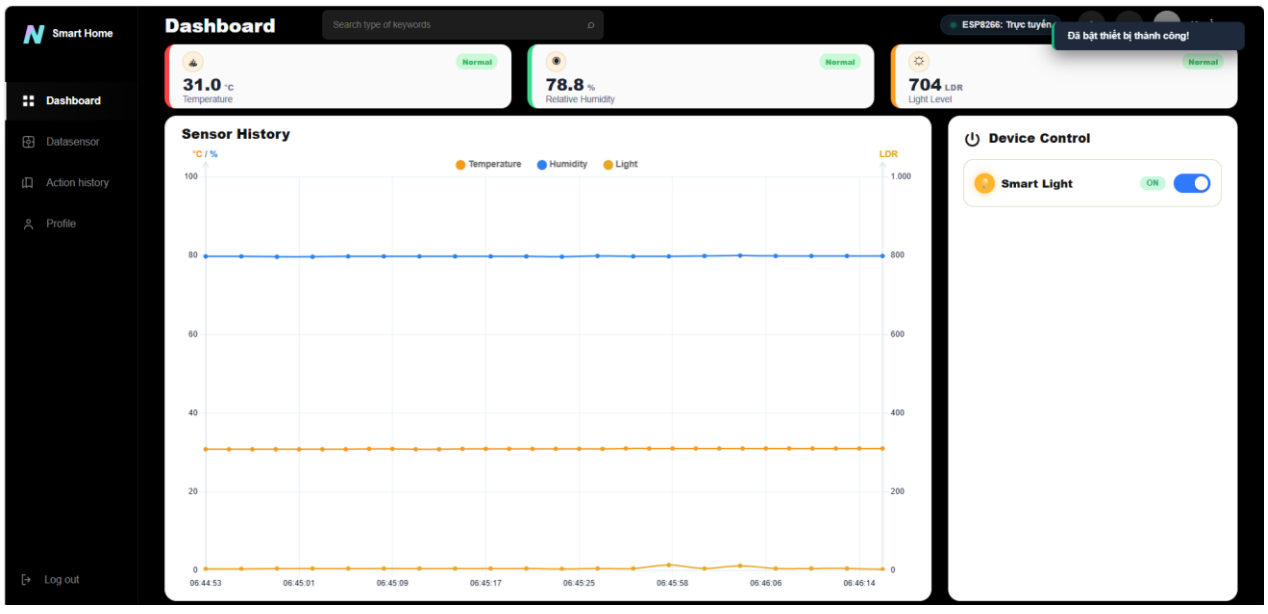
4.3 Chức năng Điều khiển thiết bị & Hiệu ứng nút xoay Loading

- **Kết quả:**
    - Tại khung "Device Control", người dùng bấm vào nút công tắc của Smart Light.
    - Ngay khi bấm, nút công tắc chuyển sang trạng thái chờ với **hiệu ứng nút xoay tròn Loading (.switch.loading)**, nhãn trạng thái chuyển thành **"Đang xử lý..."**, đồng thời ghi một bản ghi trạng thái Đang xử lý vào bảng action_history trên MySQL và gửi lệnh MQTT led1 on/led1 off xuống ESP8266.

- Sau đúng thời gian phản hồi (2 giây) của phần cứng, nút công tắc chuyển sang màu xanh **ON** (hoặc xám **OFF**), biểu tượng bóng đèn sáng lên, bản ghi trong MySQL cập nhật thành Thành công và có thông báo Popup Toast thông báo hoàn thành.



Hình 19 Hiệu ứng nút xoay Loading khi đang điều khiển thiết bị

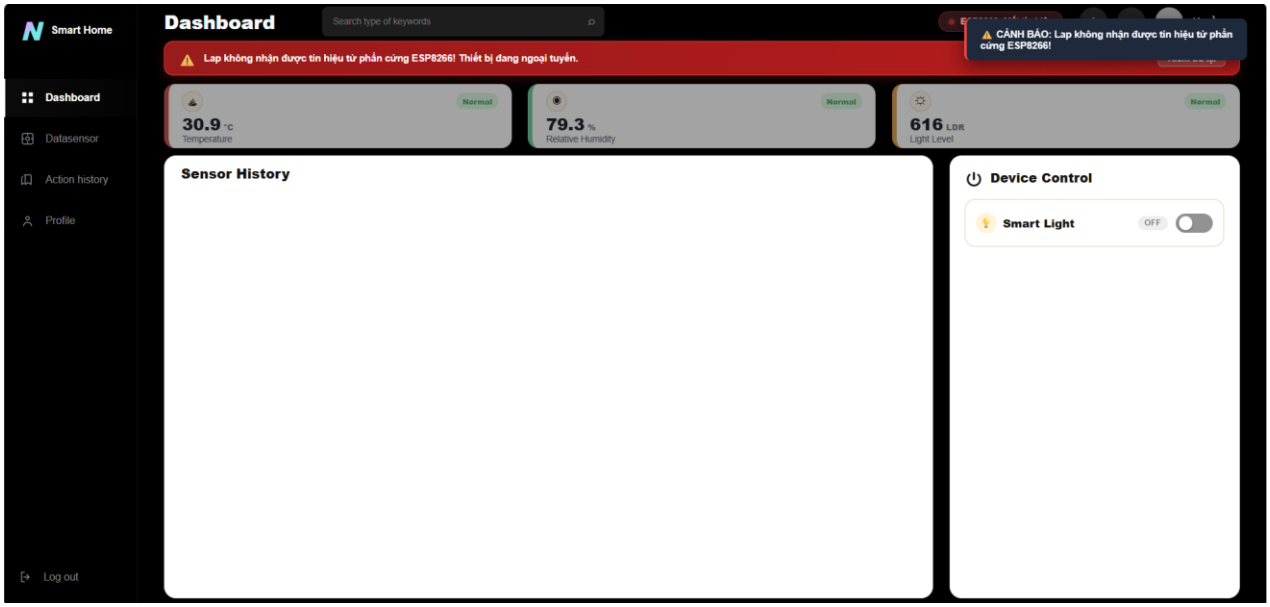


Hình 20 Trạng thái thiết bị sau khi chuyển mạch thành công

4.4 Cơ chế Cảnh báo mất tín hiệu phần cứng (Hardware Watchdog)

- **Kết quả:**
  - Server tích hợp bộ đếm thời gian Hardware Watchdog Timer. Nếu máy tính không nhận được bất kỳ tín hiệu (Heartbeat/Datasensor) nào từ ESP8266 trong quá 7 giây, hệ thống sẽ kích hoạt trạng thái Offline.

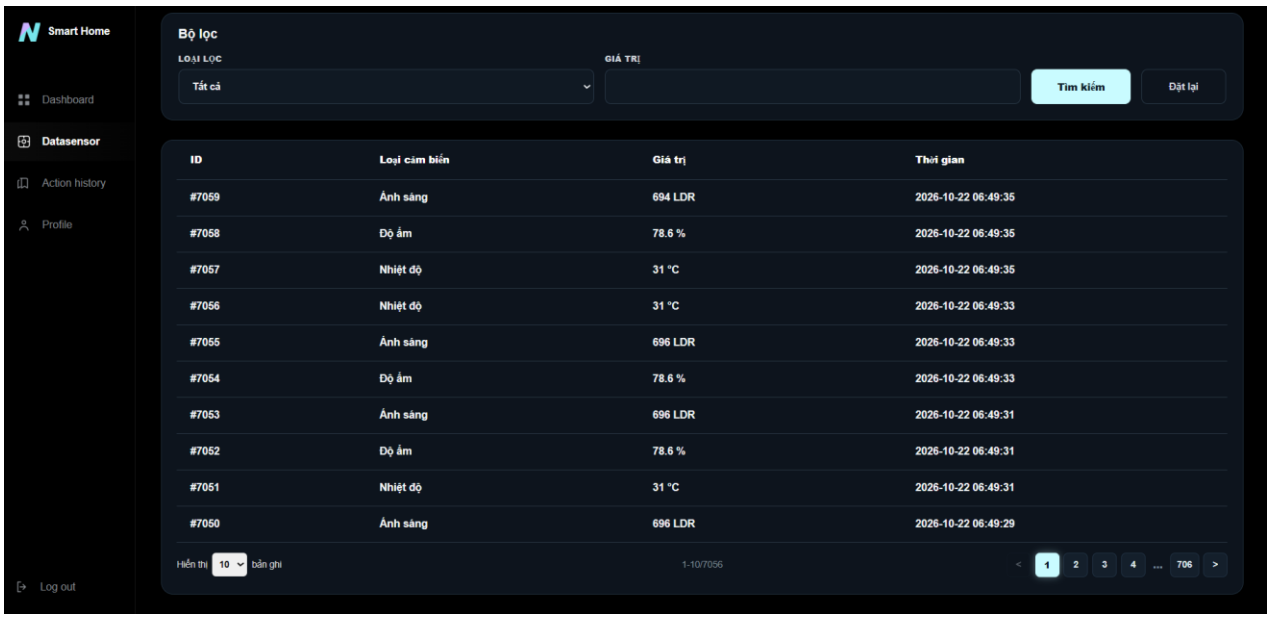
- Trên màn hình Dashboard, nhãn trạng thái chuyển thành màu đỏ **"ESP8266: Mất tín hiệu"**, xuất hiện Banner cảnh báo màu đỏ ở đầu trang: *"CẢNH BÁO: Lap không nhận được tín hiệu từ phần cứng ESP8266! Thiết bị đang ngoại tuyến."* và các thẻ thông số cảm biến bị mờ đi.
- Khi ESP8266 kết nối lại, hệ thống tự động ẩn banner cảnh báo và khôi phục nhãn xanh **"ESP8266: Trực tuyến"**.



Hình 21 Banner cảnh báo trên màn hình khi ngắt kết nối phần cứng])

4.5 Quản lý Dữ liệu cảm biến (DataSensor) với Phân trang thông minh

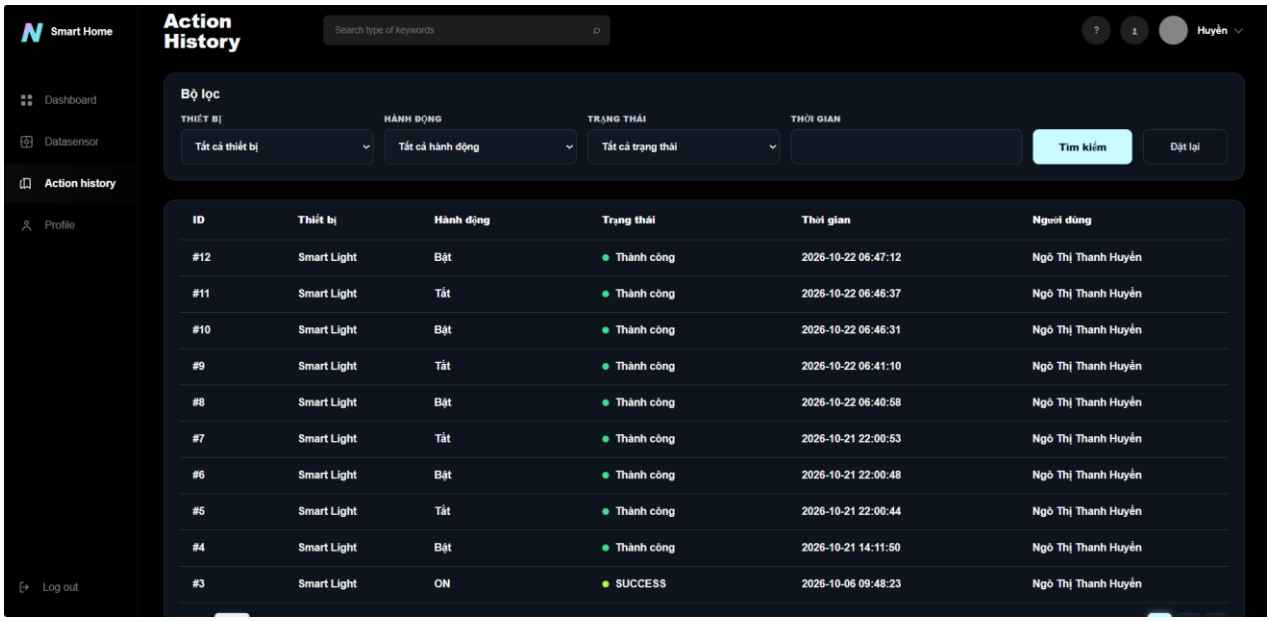
- **Kết quả:**
  - Hiển thị danh sách số đo cảm biến được đọc từ bảng datasensor trong MySQL.
  - Cung cấp các bộ lọc chuyên sâu theo: Loại cảm biến (Nhiệt độ, Độ ẩm, Ánh sáng), ô tìm kiếm giá trị/thời gian và tùy chọn số bản ghi trên mỗi trang (1, 5, 10).
  - **Thanh phân trang thu gọn thông minh:** Do dữ liệu có hơn 1.000 bản ghi, thanh phân trang được thiết kế với dấu ba chấm chuẩn dạng < 1 2 3 4 ... 10 >, nút chọn trang hiện tại có màu xanh nhạt bo tròn đẹp mắt, không bao giờ bị tràn ngoài màn hình.



Hình 22 Bảng dữ liệu DataSensor và thanh phân trang thu gọn dạng ...])

4.6 Quản lý Lịch sử thao tác (Action History)

- **Kết quả:** Bảng hiển thị đầy đủ nhật ký mọi lần bật/tắt thiết bị được lưu từ MySQL, gồm: Mã thao tác, Tên thiết bị, Hành động (Bật/Tắt), Trạng thái (Thành công / Đang xử lý có chấm trạng thái màu), Thời gian thực thi và Tên người thao tác. Có đầy đủ bộ lọc theo thiết bị, hành động và thời gian.

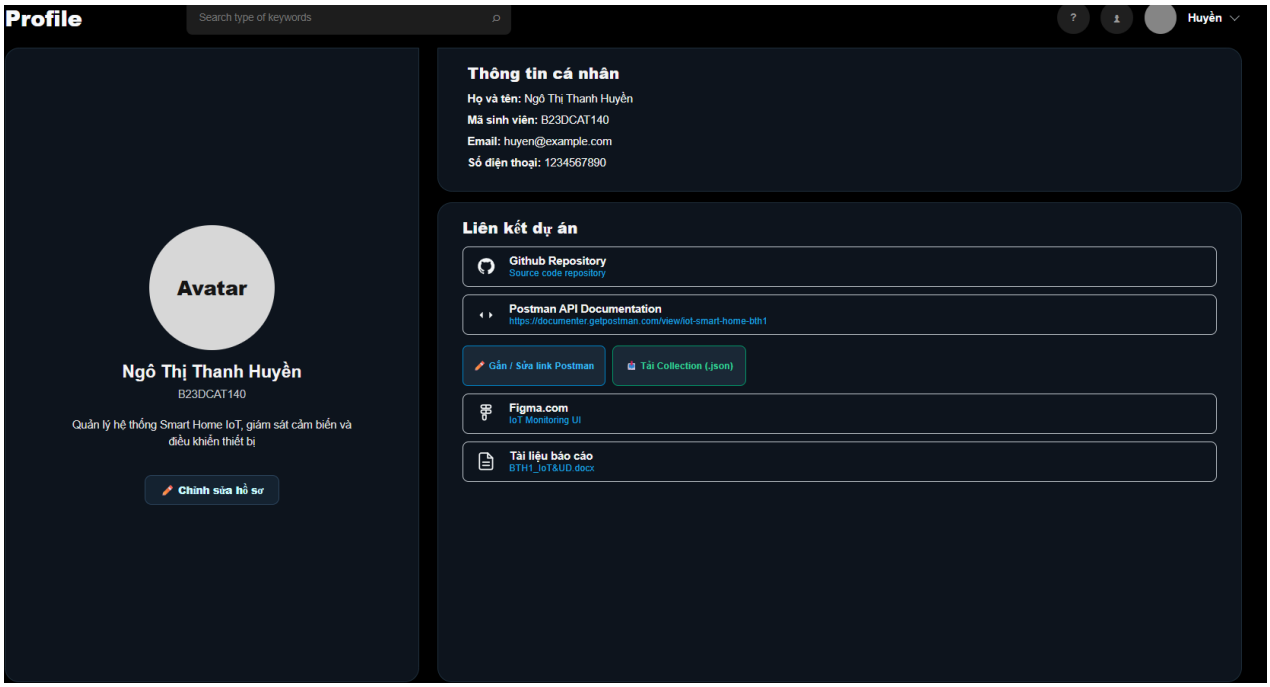


Hình 23 Bảng nhật ký điều khiển Action History

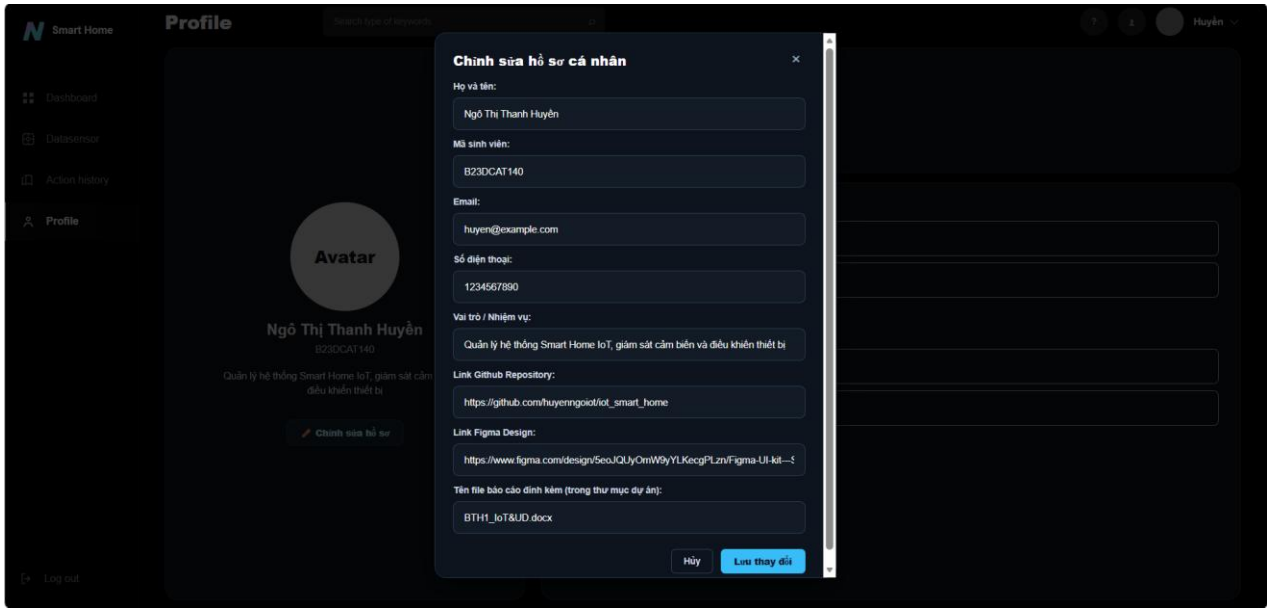
4.7 Trang Hồ sơ cá nhân (Profile) & Tích hợp liên kết

- **Kết quả:**
    - Hiển thị thông tin sinh viên thực hiện: Họ tên, Mã sinh viên, Email, Số điện thoại và Vai trò.

- **Đổi ảnh đại diện (Avatar):** Người dùng nhấp trực tiếp vào ảnh đại diện để chọn ảnh từ máy tính, ảnh được hiển thị vừa vặn trong khung tròn và lưu vào CSDL.
- **Nút Chỉnh sửa hồ sơ:** Hỗ trợ mở Popup để cập nhật thông tin cá nhân lưu trực tiếp vào bảng users trong MySQL.
- **Tích hợp liên kết sản phẩm:** Chứa các đường link tới Github Repository, Figma Design, tải trực tiếp file báo cáo BTH1_IoT&UD.docx, và mở tài liệu API Postman Documentation.



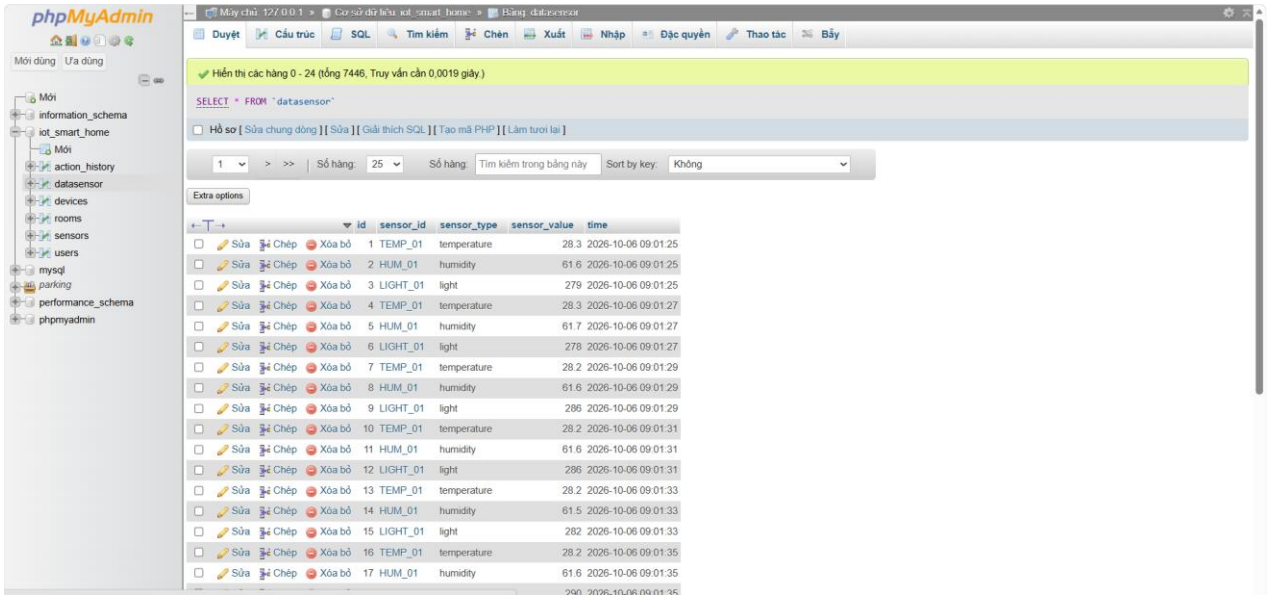
Hình 24 Giao diện trang Profile cá nhân])



Hình 25 Popup chỉnh sửa thông tin cá nhân và gắn link Postman])

4.8 Cơ sở dữ liệu MySQL trên XAMPP

- **Kết quả:** Dữ liệu cảm biến đọc từ ESP8266 và lịch sử điều khiển từ Web được lưu trữ chuẩn hóa trên 5 bảng của MySQL (datasensor, action_history, devices, sensors, users).



Hình 26 Giao diện phpMyAdmin trên XAMPP thể hiện dữ liệu trong bảng datasensor và action_history

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Qua quá trình nghiên cứu, thiết kế và triển khai đề tài "Xây dựng hệ thống nhà thông minh ứng dụng công nghệ IoT", đề tài đã vận dụng các kiến thức về phần cứng nhúng, truyền thông mạng, lập trình Web và cơ sở dữ liệu để xây dựng mô hình giám sát môi trường và điều khiển thiết bị thông qua mạng cục bộ.

Về mặt thiết kế, hệ thống được tổ chức thành các thành phần chính gồm thiết bị phần cứng, lớp truyền thông, máy chủ xử lý và giao diện người dùng. Cách tổ chức này giúp phân chia nhiệm vụ giữa các thành phần, tạo điều kiện thuận lợi cho việc kiểm thử, bảo trì và mở rộng hệ thống.

Về mặt phần cứng, ESP8266 được sử dụng làm bộ điều khiển trung tâm để tiếp nhận dữ liệu từ cảm biến nhiệt độ – độ ẩm DHT và cảm biến ánh sáng LDR. Dữ liệu thu thập được là đầu vào cho chức năng giám sát các điều kiện môi trường. Thiết bị đồng thời tiếp nhận lệnh điều khiển để thực hiện thao tác bật hoặc tắt đèn theo yêu cầu của người dùng.

Về mặt truyền thông và xử lý dữ liệu, giao thức MQTT được sử dụng để trao đổi thông tin giữa thiết bị phần cứng và máy chủ. Cơ chế này cho phép phân chia luồng dữ liệu cảm biến và luồng lệnh điều khiển, hỗ trợ quá trình cập nhật thông số cũng như phản hồi trạng

thái thao tác của thiết bị. Bên cạnh đó, các API giúp giao diện Web trao đổi dữ liệu với máy chủ để phục vụ việc hiển thị thông tin và thực hiện các chức năng quản lý.

Về mặt lưu trữ, cơ sở dữ liệu MySQL được thiết kế để quản lý thông tin tài khoản, danh mục cảm biến, thiết bị, dữ liệu đo và lịch sử điều khiển. Việc lưu trữ tập trung giúp người dùng có thể tra cứu dữ liệu đã ghi nhận, theo dõi lịch sử thao tác và hỗ trợ kiểm tra hoạt động của hệ thống.

Về mặt giao diện, ứng dụng Web được xây dựng với các chức năng chính gồm đăng nhập, Dashboard giám sát môi trường, tra cứu dữ liệu cảm biến, điều khiển thiết bị, xem lịch sử thao tác và quản lý thông tin cá nhân. Dashboard tập trung hiển thị nhiệt độ, độ ẩm và ánh sáng, đồng thời cung cấp biểu đồ trực quan để người dùng theo dõi sự thay đổi của các thông số. Chức năng điều khiển thiết bị có thể kết hợp trạng thái chờ xử lý và phản hồi kết quả nhằm giúp người dùng nhận biết tiến trình thực hiện lệnh.

Ngoài các chức năng giám sát và điều khiển, đề tài còn xây dựng cơ chế theo dõi tín hiệu từ phần cứng. Khi hệ thống không nhận được dữ liệu trong khoảng thời gian quy định, máy chủ có thể xác định tình trạng mất tín hiệu và thông báo thiết bị ngoại tuyến trên giao diện. Đây là chức năng cần thiết để người dùng nhận biết sự cố kết nối trong quá trình vận hành.

Bên cạnh những kết quả đạt được, quá trình triển khai cũng cho thấy hệ thống IoT phụ thuộc vào độ ổn định của mạng Wi-Fi, kết nối MQTT và tính hợp lệ của dữ liệu truyền nhận. Các tình huống mất kết nối, gián đoạn dữ liệu hoặc dữ liệu không đúng định dạng có thể ảnh hưởng đến trạng thái thiết bị và việc cập nhật giao diện. Vì vậy, cần tiếp tục kiểm thử và hoàn thiện cơ chế kết nối lại, kiểm tra dữ liệu đầu vào và đồng bộ trạng thái giữa phần cứng, máy chủ và cơ sở dữ liệu.

Nhìn chung, đề tài đã xây dựng được cơ sở về thiết kế và triển khai một mô hình nhà thông minh có khả năng giám sát môi trường, lưu trữ dữ liệu và hỗ trợ điều khiển thiết bị thông qua Web. Quá trình thực hiện giúp củng cố kiến thức chuyên môn, đồng thời làm rõ những yêu cầu thực tế khi tích hợp phần cứng IoT với hệ thống phần mềm. Các kết quả và kinh nghiệm thu được là nền tảng để tiếp tục hoàn thiện và mở rộng hệ thống trong thời gian tới.

5.2 Hạn chế của đề tài

Mặc dù đã xây dựng được các thành phần và chức năng chính, hệ thống vẫn còn một số hạn chế cần được khắc phục.

Thứ nhất, về kết nối và truyền dữ liệu: Hệ thống phụ thuộc vào mạng Wi-Fi và kết nối MQTT giữa ESP8266 với máy chủ. Khi kết nối bị gián đoạn, dữ liệu cảm biến có thể không được gửi đến máy chủ đúng chu kỳ, ảnh hưởng đến tính liên tục của quá trình giám sát. Vì vậy, cần tiếp tục cải thiện cơ chế tự động kết nối lại và kiểm tra trạng thái kết nối.

Thứ hai, về độ chính xác của dữ liệu cảm biến: Các giá trị đo phụ thuộc vào điều kiện môi trường, chất lượng cảm biến, cách đấu nối và phương pháp đọc dữ liệu. Đặc biệt, giá trị từ quang trở LDR chủ yếu phản ánh mức tín hiệu ánh sáng tương đối, chưa phải phép đo cường độ sáng theo đơn vị vật lý đã hiệu chuẩn.

Thứ ba, về khả năng mở rộng: Mô hình hiện tập trung vào một thiết bị chiếu sáng và một số thông số môi trường cơ bản. Khi bổ sung nhiều thiết bị hoặc nhiều khu vực trong nhà, hệ thống cần được mở rộng về cấu trúc quản lý thiết bị, cách tổ chức các chủ đề MQTT, giao diện điều khiển và cơ chế phân quyền.

Thứ tư, về khả năng vận hành: Hệ thống cần được kiểm thử thêm với nhiều tình huống khác nhau như mất điện, mất Wi-Fi, máy chủ khởi động lại, dữ liệu MQTT sai định dạng và thiết bị không phản hồi. Những thử nghiệm này giúp đánh giá khả năng phục hồi và tính nhất quán của dữ liệu khi xảy ra sự cố.

Thứ năm, về bảo mật: Khi mở rộng phạm vi sử dụng, cần tăng cường bảo vệ thông tin đăng nhập, kiểm soát quyền truy cập các API, bảo vệ thông tin xác thực MQTT và sử dụng kênh truyền được mã hóa khi phù hợp. Đây là các yêu cầu quan trọng để hạn chế nguy cơ truy cập trái phép và thao tác điều khiển không hợp lệ.

5.3 Hướng phát triển trong tương lai

Trong thời gian tới, hệ thống có thể được phát triển theo các hướng sau:

Thứ nhất, mở rộng hệ thống phần cứng**:** Bổ sung nhiều thiết bị chiếu sáng, quạt và các thiết bị điện khác; đồng thời tích hợp thêm cảm biến phát hiện chuyển động, khói hoặc khí gas. Hệ thống có thể được mở rộng để quản lý nhiều phòng, trong đó mỗi thiết bị được định danh riêng nhằm thuận tiện cho việc giám sát và điều khiển.

Thứ hai, phát triển chức năng tự động hóa**:** Xây dựng các kịch bản điều khiển dựa trên dữ liệu cảm biến. Ví dụ, hệ thống có thể tự động bật đèn khi mức ánh sáng xuống thấp hoặc kích hoạt quạt khi nhiệt độ vượt ngưỡng cài đặt. Các ngưỡng điều khiển cần được cấu hình linh hoạt để phù hợp với điều kiện sử dụng thực tế.

Thứ ba, nâng cao độ ổn định của truyền thông**:** Hoàn thiện cơ chế tự động kết nối lại Wi-Fi và MQTT, kiểm tra tính hợp lệ của bản tin, xử lý dữ liệu lỗi và đồng bộ trạng thái thiết bị sau khi kết nối được khôi phục. Ngoài ra, có thể bổ sung cơ chế lưu đệm dữ liệu khi mất kết nối và gửi bù khi mạng hoạt động trở lại, nhằm hạn chế thất thoát thông tin.

Thứ tư, hoàn thiện chức năng quản lý dữ liệu: Bổ sung các công cụ thống kê, lọc dữ liệu theo thời gian, xuất báo cáo và trực quan hóa xu hướng thay đổi của các thông số môi trường. Việc tối ưu truy vấn và phân trang cũng cần được tiếp tục thực hiện khi số lượng bản ghi tăng lên.

Thứ năm, nâng cao tính bảo mật: Hoàn thiện cơ chế xác thực và phân quyền người dùng, bảo vệ các API điều khiển thiết bị, quản lý thông tin xác thực an toàn và xem xét triển khai mã hóa dữ liệu trong quá trình truyền thông. Đồng thời, cần ghi nhận các thao tác quan trọng để hỗ trợ kiểm tra và truy vết khi xảy ra sự cố.

Thứ sáu, phát triển ứng dụng trên thiết bị di động: Xây dựng ứng dụng dành cho điện thoại thông minh, cho phép người dùng theo dõi thông số môi trường, nhận thông báo khi thiết bị ngoại tuyến và điều khiển thiết bị thuận tiện hơn mà không phụ thuộc hoàn toàn vào giao diện trình duyệt trên máy tính.

Tóm lại, việc tiếp tục phát triển theo các hướng trên sẽ giúp hệ thống nâng cao độ ổn định, khả năng mở rộng, tính bảo mật và mức độ tự động hóa. Đây là cơ sở để từng bước hoàn thiện mô hình thử nghiệm thành một hệ thống nhà thông minh linh hoạt, thuận tiện và đáp ứng tốt hơn các nhu cầu sử dụng thực tế.